LAPORAN PRAKTIKUM TUGAS AKHIR PENGEMBANGAN APLIKASI BERBASIS WEB

Modul 9: HMVC, Authorization, dan AJAX
dengan React pada Laravel



Disusun Oleh:

Nama: Dryhus Dzacky Damingtyas.S

NIM: 707012530010

Kelas: D4SIKC-49-04

Mata Kuliah: Pengembangan Aplikasi Berbasis Web

Dosen Pengampu: .....

UNIVERSITAS TELKOM
FAKULTAS ILMU TERAPAN
D4 SISTEM INFORMASI KOTACERDAS

TAHUN AKADEMIK 2026/2027

A. Tujuan Praktikum

Praktikum ini saya kerjakan untuk membangun aplikasi penerimaan mahasiswa baru pada project `pertemuan9_hmvc_react` dengan arsitektur HMVC memakai React Starter Kit. Berbeda dengan Modul 8 yang menulis tampilan dengan Blade dan jQuery, seluruh halaman modul pada praktikum ini ditulis dengan React dan dihubungkan ke Laravel melalui Inertia.js. Modul UserManagement dikerjakan dengan CRUD Inertia, sedangkan modul Pendaftaran dikerjakan dengan CRUD AJAX serta upload berkas. Hak akses setiap halaman dibatasi oleh role admin dan user yang diperiksa middleware di sisi server.

Secara lebih rinci, tujuan yang ingin dicapai pada praktikum ini adalah:

1. Menyiapkan project Laravel React Starter Kit, memasang paket `nwidart/laravel-modules` beserta merge-plugin pada `composer.json`, dan membuat dua modul HMVC untuk fitur UserManagement dan Pendaftaran.
2. Mengatur agar Inertia dapat menemukan halaman React yang berada di dalam folder modul melalui fungsi `resolve` pada `app.tsx`, serta mendaftarkan folder modul pada Tailwind CSS dan `tsconfig.json`.
3. Menambahkan kolom role pada tabel `users`, membuat enum `UserRole`, factory, dan seeder, lalu menyesuaikan proses register agar akun hasil register selalu berperan user.
4. Membagikan data auth dan flash ke seluruh halaman React sebagai shared props, lalu membuat hook `useSharedProps` serta komponen `AlertMessage`, `FlashMessage`, dan `ConfirmDialog`.
5. Membuat middleware `role`, mendaftarkan aliasnya pada `bootstrap/app.php`, dan menampilkan menu sidebar sesuai role pengguna.
6. Membangun modul UserManagement dengan CRUD Inertia khusus admin, lengkap dengan pencarian, filter role, pagination, dan aturan bahwa admin tidak dapat menurunkan role atau menghapus akunnya sendiri.
7. Membangun modul Pendaftaran dengan CRUD AJAX memakai `fetch` dan `FormData`, termasuk upload berkas yang dapat dibuka kembali, diganti saat edit, dan ikut terhapus dari storage.
8. Menampilkan error validasi 422 di bawah setiap input pada form AJAX, serta pesan berhasil setiap kali data ditambah, diubah, atau dihapus.
9. Membuktikan pembatasan akses (error 403), perilaku AJAX pada tab Network, isi folder storage, dan hasil pengujian otomatis Pest.

B. Alat dan Bahan

Praktikum ini dikerjakan pada satu laptop dengan spesifikasi perangkat keras dan perangkat lunak sebagai berikut.

1. Perangkat Keras

- Laptop dengan prosesor 13th Gen Intel Core i7-13620H.
- RAM 16 GB (terbaca 15,7 GB).
- Ruang penyimpanan kosong lebih dari 3 GB untuk folder `node_modules` dan `vendor`.
- Sistem operasi Windows 11 Home Single Language build 10.0.26200.

2. Perangkat Lunak

No	Perangkat Lunak	Versi / Keterangan
1	XAMPP	8.2.12, berisi Apache 2.4.58, MariaDB 10.4.32, PHP 8.2.12, dan phpMyAdmin 5.2.1
2	Composer	2.10.3 sebagai dependency manager PHP
3	Laravel Framework	12.71.0 dengan React Starter Kit
4	React dan Inertia.js	React 19.0.0 dan @inertiajs/react 2.0.3
5	Inertia Laravel	inertiajs/inertia-laravel 2.0.28
6	Tailwind CSS	4.0.8 beserta shadcn/ui
7	Laravel Fortify	autentikasi bawaan starter kit
8	nwidart/laravel-modules	12.0.5 sebagai paket arsitektur HMVC
9	Node.js dan npm	Node.js 24.13.1 dan npm 11.5.2, mengompilasi aset melalui Vite 6.1.1
10	TypeScript	5.7.3
11	Pest	3.8.7 sebagai framework pengujian otomatis
12	Visual Studio Code	editor teks
13	Google Chrome	browser dan alat pemeriksaan tab Network pada DevTools
14	Git	version control

Database yang dipakai adalah MariaDB dengan nama `hmvc_react`, sedangkan aplikasi dijalankan pada alamat `http://127.0.0.1:8010` karena port 8000 sedang dipakai proses lain.

C. Langkah-Langkah Praktikum

Aplikasi saya bangun pada project `pertemuan9_hmvc_react` dengan database `hmvc_react`. Autentikasi memakai fitur bawaan React Starter Kit, sedangkan dua modul HMVC diisi sesuai tema penerimaan mahasiswa baru. Koneksi database dan locale diatur pada file `.env`, yaitu `DB_CONNECTION=mysql` dan `DB_DATABASE=hmvc_react` agar data disimpan di MariaDB pada XAMPP, bukan di SQLite bawaan.

C.1 Menyiapkan Project dan Memasang HMVC

Project dibuat dengan Laravel Installer memakai starter kit React, autentikasi bawaan, dan Pest sebagai framework pengujian. Setelah project terbentuk, saya memasang paket HMVC lalu mengizinkan plugin merge Composer. Blok `merge-plugin` saya tambahkan pada bagian `extra` di `composer.json` agar Composer ikut membaca `composer.json` milik setiap modul, sehingga kelas di dalam folder `Modules` ter-autoload tanpa konfigurasi PSR-4 manual. Sesudah mengubah `composer.json`, saya menjalankan `composer dump-autoload` supaya peta kelas diperbarui.



```
{
    "require": {
        "php": "^8.2",
        "inertiajs/inertia-laravel": "^2.0",
        "laravel/framework": "^12.0",
        "laravel/tinker": "^2.10.1",
        "nwidart/laravel-modules": "^12.0",
        "tightenco/ziggy": "^2.4"
    },
    "extra": {
        "laravel": { "dont-discover": [] },
        "merge-plugin": {
            "include": ["Modules/*/composer.json"]
        }
    },
    "config": {
        "optimize-autoloader": true,
        "preferred-install": "dist",
        "sort-packages": true,
        "allow-plugins": {
            "pestphp/pest-plugin": true,
            "php-http/discovery": true,
            "wikimedia/composer-merge-plugin": true
        }
    }
}
```

Kode 1. Bagian require, merge-plugin, dan allow-plugins pada composer.json.

Dua modul saya buat sekaligus dengan perintah `php artisan module:make UserManagement Pendaftaran`. Karena tampilan modul ditulis dengan React, folder `resources/views`, `resources/assets`, `vite.config.js`, dan `package.json` bawaan modul saya biarkan tidak dipakai, lalu saya buat folder `resources/js/pages` di dalam tiap modul. Route API bawaan modul yang memakai middleware `auth:sanctum` saya kosongkan karena Sanctum tidak dipasang pada project ini. Status kedua modul dapat diperiksa pada dua berkas berikut.

```
// modules_statuses.json
{
        "UserManagement": true,
        "Pendaftaran": true
}

// Modules/Pendaftaran/routes/api.php
<?php
// modul ini tidak menyediakan endpoint API
```

Kode 2. modules_statuses.json dan route API modul yang dikosongkan.

Middleware `role` didaftarkan pada `bootstrap/app.php`. Laravel versi ini tidak lagi memakai `app/Http/Kernel.php`, sehingga alias middleware ditulis di dalam `withMiddleware`. Selain alias `role`, pada blok yang sama tetap dipertahankan middleware bawaan starter kit, yaitu `HandleInertiaRequests` dan `AddLinkHeadersForPreloadedAssets`.

```
<?php

use App\Http\Middleware\HandleInertiaRequests;
use App\Http\Middleware\EnsureUserHasRole;
use Illuminate\Foundation\Application;
use Illuminate\Http\Middleware\AddLinkHeadersForPreloadedAssets;

return Application::configure(basePath: dirname(__DIR__))
        ->withRouting(
                web: __DIR__ . '/../routes/web.php',
                commands: __DIR__ . '/../routes/console.php',
                health: '/up',
        )
        ->withMiddleware(function (Middleware $middleware): void {
                $middleware->encryptCookies(except: ['appearance', 'sidebar_state']);

                $middleware->web(append: [
                        HandleInertiaRequests::class,
                        AddLinkHeadersForPreloadedAssets::class,
                ]);

                $middleware->alias([
                        'role' => EnsureUserHasRole::class,
                ]);

        })
        ->withExceptions(function (Exceptions $exceptions) {
                //
        })->create();
```

Kode 3. Alias middleware role pada bootstrap/app.php.

C.2 Mengatur Halaman React di Dalam Modul

Agar Inertia dapat menemukan halaman milik modul, bagian `resolve` pada `createInertiaApp` saya ubah. Vite mengumpulkan seluruh halaman React dari `resources/js/pages` dan `Modules/*/resources/js/pages` melalui `import.meta.glob`. Jika nama halaman memuat tanda `::`, misalnya `Pendaftaran::index`, nama itu dipecah menjadi nama modul dan nama halaman, lalu berkas `Modules/Pendaftaran/resources/js/pages/index.tsx` dimuat oleh `resolvePageComponent`. Tanpa tanda tersebut, halaman dicari di `resources/js/pages` seperti biasa.

```tsx
import '../css/app.css';
import { createInertiaApp } from '@inertiajs/react';
import { resolvePageComponent } from 'laravel-vite-plugin/inertia-helpers';

createInertiaApp({
        title: (title) => (title ? `${title} - ${appName}` : appName),

        resolve: (name) => {
                const pages = import.meta.glob([
                        './pages/**/*.tsx',
                        '../../Modules/*/resources/js/pages/**/*.tsx',
                ]);

                // Halaman modul ditulis dengan format NamaModul::nama-halaman
                if (name.includes('::')) {
                        const [module, page] = name.split('::');
                        return resolvePageComponent(
                                `../../Modules/${module}/resources/js/pages/${page}.tsx`,
                                pages
                        );
                }

                return resolvePageComponent(`./pages/${name}.tsx`, pages);
        },
        /* ... setup() dan progress() ... */
});
```

Kode 4. Pengaturan resolve halaman modul pada app.tsx.

Folder modul juga saya daftarkan pada Tailwind CSS lewat baris `@source` di `app.css` agar class Tailwind yang ditulis di halaman modul ikut dikompilasi. Tanpa baris ini, halaman modul tampil tanpa style karena Tailwind 4 hanya memindai berkas yang disebutkan secara eksplisit. Pada `tsconfig.json` saya menambahkan pola `Modules/*/resources/js/**/*.ts` dan `*.tsx` ke bagian `include` supaya Visual Studio Code mengenali TypeScript di dalam modul.

```
/* resources/css/app.css */
@import 'tailwindcss';
@plugin 'tailwindcss-animate';

@source '../../vendor/laravel/framework/src/Illuminate/Pagination/resources/views/*.blade.php';
@source '../../Modules/*/resources/js/**/*.{ts,tsx}';

/* tsconfig.json */
{
    "include": [
        "resources/js/**/*.ts",
        "resources/js/**/*.d.tsx",
        "resources/js/**/*.tsx",
        "Modules/*/resources/js/**/*.ts",
        "Modules/*/resources/js/**/*.d.tsx"
    ]
}
```

Kode 5. Baris @source pada app.css dan pola include pada tsconfig.json.

C.3 Menambahkan Role pada Tabel Users

Kolom role saya tambahkan lewat migration baru supaya tabel `users` bawaan Laravel tidak diubah langsung. Nilai bawaan kolom ini adalah `user`, sehingga akun yang tidak menyebut role otomatis menjadi user biasa. Nilai role saya bungkus dalam backed enum `UserRole` dengan dua case, yaitu `Admin` dan `User`, beserta method `label()` untuk menampilkan nama role yang mudah dibaca. Karena nilainya enum, salah ketik role tidak mungkin lolos ke database.

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
        public function up(): void
        {
                Schema::table('users', function (Blueprint $table) {
                        $table->string('role')->default('user')->after('email');
                });
        }

        public function down(): void
        {
                Schema::table('users', function (Blueprint $table) {
                        $table->dropColumn('role');
                });
        }
};
```

Kode 6. Migration penambah kolom role pada tabel users.

```php
<?php

namespace App\Enums;

enum UserRole: string
{
        case Admin = 'admin';
        case User = 'user';

        /**
          * Get the human-readable label of the role.
          */
        public function label(): string
        {
                return match ($this) {
                        self::Admin => 'Admin',
                        self::User => 'User',
                };
        }
}
```

Kode 7. Enum UserRole dengan dua case dan method label().

Model `User` saya lengkapi dengan cast `role` ke enum, nilai bawaan `role` sebesar `user`, serta dua method bantu yaitu `isAdmin()` dan `hasRole()`. Method `isAdmin()` dipakai controller, form request, dan halaman React, sedangkan `hasRole()` dipakai middleware `role` karena menerima daftar role sebagai argumen variadik. Method `hasRole()` juga mengubah string dari route menjadi enum lewat `UserRole::tryFrom()`, sehingga role yang tidak dikenal akan dianggap tidak cocok.

```php
<?php

namespace App\Models;

use App\Enums\UserRole;
use Database\Factories\UserFactory;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Foundation\Auth\User as Authenticatable;
use Illuminate\Notifications\Notifiable;

class User extends Authenticatable
{
        /** @use HasFactory<UserFactory> */
        use HasFactory, Notifiable;

        /**
          * The attributes that are mass assignable.
          *
          * @var list<string>
          */
        protected $fillable = [
                'name',
                'email',
                'role',
                'password',
        ];

        /**
          * The attributes that should be hidden for serialization.
          *
          * @var list<string>
          */
        protected $hidden = [
                'password',
                'remember_token',
        ];

        /**
          * The model's default attribute values.
          *
          * @var array<string, mixed>
          */
        protected $attributes = [
                'role' => 'user',
        ];

        /**
          * Get the attributes that should be cast.
          *
          * @return array<string, string>
          */
        protected function casts(): array
        {
                return [
                        'email_verified_at' => 'datetime',
                        'password' => 'hashed',
                        'role' => UserRole::class,
                ];
        }

        public function isAdmin(): bool
        {
                return $this->role === UserRole::Admin;
        }

        public function hasRole(UserRole|string ...$roles): bool
        {
                foreach ($roles as $role) {
                        $targetRole = $role instanceof UserRole ? $role : UserRole::tryFrom($

                        if ($this->role === $targetRole) {
                                return true;
                        }
                }

                return false;
        }
}
```

Kode 8. Model User dengan cast role dan method bantu isAdmin() serta hasRole().

Factory diberi state `admin()` supaya pembuatan akun admin di seeder dan pengujian cukup memanggil `User::factory()->admin()->create()`. State ini hanya mengisi kolom role, sehingga atribut lain tetap memakai nilai bawaan factory.

```
        public function definition(): array
        {
                return [
                        'name' => fake()->name(),
                        'email' => fake()->unique()->safeEmail(),
                        'role' => UserRole::User,
                        'email_verified_at' => now(),
                        'password' => static::$password ??= Hash::make('password'),
                        'remember_token' => Str::random(10),
                ];
        }

        /**
          * Indicate that the user is an admin.
          */
        public function admin(): static
        {
                return $this->state(fn (array $attributes) => [
                        'role' => UserRole::Admin,
                ]);
        }

        /**
          * Indicate that the user's email address is not verified.
          */
        public function unverified(): static
        {
                return $this->state(fn (array $attributes) => [
                        'email_verified_at' => null,
                ]);
        }
}
```

Kode 9. UserFactory dengan state admin().

Seeder membuat satu akun admin `admin@example.com` dan satu akun user `user@example.com`, keduanya dengan password `password`. Keduanya dibuat lewat `firstOrCreate`, sehingga menjalankan seeder dua kali tidak menggandakan akun.

```php
        public function run(): void
        {
                $admin = User::factory()->admin()->make([
                        'name' => 'Administrator',
                        'password' => 'password',
                ])->forceFill(['email' => 'admin@example.com'])->getAttributes();

                User::firstOrCreate(['email' => 'admin@example.com'], $admin);

                $user = User::factory()->make([
                        'name' => 'User Biasa',
                        'password' => 'password',
                ])->forceFill(['email' => 'user@example.com'])->getAttributes();

                User::firstOrCreate(['email' => 'user@example.com'], $user);
        }
}
```

Kode 10. DatabaseSeeder yang membuat akun admin dan user.

Proses register memakai `RegisteredUserController` bawaan starter kit. Akun hasil register selalu diberi role user karena field `role` tidak pernah diisi dari input pengguna, sehingga pengguna tidak dapat mendaftarkan dirinya sebagai admin. Akun admin hanya dibuat lewat seeder atau oleh admin pada modul UserManagement.

```php
        public function store(Request $request): RedirectResponse
        {
                $request->validate([
                        'name' => 'required|string|max:255',
                        'email' => 'required|string|lowercase|email|max:255|unique:'.User::cl
                        'password' => ['required', 'confirmed', Rules\Password::defaults()],
                ]);

                $user = User::create([
                        'name' => $request->name,
                        'email' => $request->email,
                        'password' => Hash::make($request->password),
                ]);

                event(new Registered($user));

                Auth::login($user);

                return to_route('dashboard');
        }
}
```

Kode 11. RegisteredUserController yang tidak pernah mengisi role dari input pengguna.

C.4 Authentication dan Shared Props

React Starter Kit sudah menyediakan halaman login, register, logout, serta reset password, jadi bagian ini tidak saya tulis ulang. Pesan galat login bawaan Laravel sudah berbahasa

Indonesia pada file `lang/id/auth.php`. Untuk menampilkan pesan berhasil dan pesan error dari redirect controller, saya menambahkan key `flash` pada method `share()` di `HandleInertiaRequests`.

```php
<?php

namespace App\Http\Middleware;

use Illuminate\Http\Request;
use Inertia\Middleware;

class HandleInertiaRequests extends Middleware
{
        protected $rootView = 'app';

        public function share(Request $request): array
        {
                [$message, $author] = str(Inspiring::quotes()->random())->explode('-');

                return array_merge(parent::share($request), [
                        ...parent::share($request),
                        'name' => config('app.name'),
                        'quote' => ['message' => trim($message), 'author' => trim($author)],
                        'auth' => [
                                'user' => $request->user(),
                        ],
                        'flash' => [
                                'success' => fn () => $request->session()->get('success'),
                                'error' => fn () => $request->session()->get('error'),
                        ]
                ]);
        }
}
```

Kode 12. Shared props auth dan flash pada HandleInertiaRequests.

Agar shared props mudah dibaca, saya membuat hook `useSharedProps` yang mengembalikan `user`, `isAdmin`, dan `flash`. Nilai `isAdmin` dipakai untuk menampilkan menu Kelola User, tetapi perlindungan yang sebenarnya tetap dilakukan oleh middleware role di sisi server. Saya juga membuat komponen `AlertMessage` dan `FlashMessage` untuk pesan, serta `ConfirmDialog` untuk dialog konfirmasi hapus yang dipakai bersama oleh kedua modul.

```ts
import { usePage } from '@inertiajs/react';

export type UserRole = 'admin' | 'user';

export type AuthUser = {
    id: number;
    name: string;
    email: string;
    role: UserRole;
};

type SharedProps = {
    auth: {
        user: AuthUser;
    };
    flash: {
        success?: string | null;
        error?: string | null;
    };
};

export function useSharedProps() {
    const { auth, flash } = usePage<SharedProps>().props;

    return {
        user: auth.user,
        isAdmin: auth.user?.role === 'admin',
        flash,
    };
}
```

Kode 13. Hook useSharedProps.

```tsx
import { CircleAlert, CircleCheck, X } from 'lucide-react';
import { useEffect, useState } from 'react';
import { useSharedProps } from '@/hooks/use-shared-props';

type AlertMessageProps = {
    type: 'success' | 'error';
    message?: string | null;
    onClose?: () => void;
};

export function AlertMessage({ type, message, onClose }: AlertMessageProps) {
    // Pesan berhasil otomatis tertutup setelah 4 detik
    useEffect(() => {
        if (!message || type !== 'success' || !onClose) {
            return;
        }
        const timer = setTimeout(onClose, 4000);
        return () => clearTimeout(timer);
    }, [message, type, onClose]);

    if (!message) {
        return null;
    }

    const Icon = type === 'success' ? CircleCheck : CircleAlert;

    const color =
        type === 'success'
            ? 'border-green-200 bg-green-50 text-green-800 dark:border-green-900 dark:bg-green-950 dark:text-green-200'
            : 'border-red-200 bg-red-50 text-red-800 dark:border-red-900 dark:bg-red-950 dark:text-red-200';

    return (
        <div role="alert" className={`flex items-start gap-3 rounded-lg border px-4 py-3 text-sm ${color}`}>
            <Icon className="mt-0.5 size-4 shrink-0" />
            <p className="flex-1">{message}</p>
            {onClose && (
                <button
                    type="button"
                    onClick={onClose}
                    className="opacity-70 hover:opacity-100"
                    aria-label="Tutup"
                >
                    <X className="size-4" />
                </button>
            )}
        </div>
    );
}
```

Kode 14. Komponen AlertMessage.

```jsx
export function FlashMessage() {
    const { flash } = useSharedProps();
    const [success, setSuccess] = useState(flash.success);
    const [error, setError] = useState(flash.error);

    // Tampilkan ulang pesan setiap kali server mengirim flash baru
    useEffect(() => {
        setSuccess(flash.success);
        setError(flash.error);
    }, [flash]);

    return (
        <>
            <AlertMessage
                type="success"
                message={success}
                onClose={() => setSuccess(null)}
            />
            <AlertMessage
                type="error"
                message={error}
                onClose={() => setError(null)}
            />
        </>
    );
}
```

Kode 15. Komponen FlashMessage.

```tsx
import { Button } from '@/components/ui/button';
import {
    Dialog,
    DialogClose,
    DialogContent,
    DialogDescription,
    DialogFooter,
    DialogHeader,
    DialogTitle,
} from '@/components/ui/dialog';

type ConfirmDialogProps = {
    open: boolean;
    title?: string;
    description: string;
    confirmLabel?: string;
    processing?: boolean;
    onConfirm: () => void;
    onOpenChange: (open: boolean) => void;
};

export function ConfirmDialog({
    open,
    title = 'Konfirmasi Hapus',
    description,
    confirmLabel = 'Ya, hapus',
    processing = false,
    onConfirm,
    onOpenChange,
}: ConfirmDialogProps) {
    return (
        <Dialog open={open} onOpenChange={onOpenChange}>
            <DialogContent>
                <DialogHeader>
                    <DialogTitle>{title}</DialogTitle>
                    <DialogDescription>{description}</DialogDescription>
                </DialogHeader>
                <DialogFooter className="gap-2">
                    <DialogClose asChild>
                        <Button variant="secondary" disabled={processing}>
                            Batal
                        </Button>
                    </DialogClose>
                    <Button variant="destructive" onClick={onConfirm} disabled={processing}>
                        {processing ? 'Menghapus...' : confirmLabel}
                    </Button>
                </DialogFooter>
            </DialogContent>
        </Dialog>
    );
}
```

Kode 16. Komponen ConfirmDialog.

Menu sidebar saya susun pada `app-sidebar.tsx`. Menu Kelola User hanya ditambahkan jika `isAdmin` bernilai true, sedangkan menu Dashboard dan Pendaftaran selalu tampil. `NavMain` bawaan starter kit otomatis memberi tanda aktif pada menu yang sedang dibuka.

```tsx
export function AppSidebar() {
        const { isAdmin } = useSharedProps();

        const mainNavItems: NavItem[] = [
                { title: 'Dashboard', url: '/dashboard', icon: LayoutGrid },
                { title: 'Pendaftaran', url: '/pendaftaran', icon: ClipboardList },
                ...(isAdmin ? [{ title: 'Kelola User', url: '/admin/users', icon: Users }] :
        ];

        return (
                <Sidebar collapsible="icon" variant="inset">
                        <SidebarHeader>
                                <SidebarMenu>
                                        <SidebarMenuItem>
                                                <SidebarMenuButton size="lg" asChild>
                                                        <Link href="/dashboard" prefetch>
                                                                <AppLogo />
                                                        </Link>
                                                </SidebarMenuButton>
                                        </SidebarMenuItem>
                                </SidebarMenu>
                        </SidebarHeader>

                        <SidebarContent>
                                <NavMain items={mainNavItems} />
                        </SidebarContent>

                        <SidebarFooter>
                                <NavUser />
                        </SidebarFooter>
                </Sidebar>
        );
}
```

Kode 17. Menu sidebar berbasis role pada app-sidebar.tsx.

Halaman dashboard dirender oleh `DashboardController` yang bersifat invokable. Controller mengirim total pendaftar dari model `Daftar` milik modul Pendaftaran, sedangkan total user hanya dikirim untuk admin dan bernilai null bagi user biasa. Halaman `dashboard.tsx` menerima kedua props itu dan hanya menampilkan kartu Total Pengguna jika `isAdmin` bernilai true.

```php
<?php

namespace App\Http\Controllers;

use App\Models\User;
use Illuminate\Http\Request;
use Inertia\Inertia;
use Inertia\Response;
use Modules\Pendaftaran\Models\Daftar;

class DashboardController extends Controller
{
        /**
          * Show the dashboard with record counts per module.
          */
        public function __invoke(Request $request): Response
        {
                return Inertia::render('dashboard', [
                        'totalPendaftar' => Daftar::count(),
                        'totalUsers' => $request->user()->isAdmin() ? User::count() : null,
                ]);
        }
}
```

Kode 18. DashboardController.

```php
<?php

use App\Http\Controllers\DashboardController;
use Illuminate\Support\Facades\Route;
use Inertia\Inertia;

Route::get('/', function () {
        return Inertia::render('welcome', [
                'canRegister' => Route::has('register'),
        ]);
})->name('home');

Route::middleware(['auth', 'verified'])->group(function () {
        Route::get('dashboard', DashboardController::class)->name('dashboard');
});

require __DIR__.'/auth.php';
require __DIR__.'/settings.php';
```

Kode 19. Route dashboard dan route halaman utama pada routes/web.php.



```tsx
export default function Dashboard({ totalPendaftar, totalUsers }: DashboardProps) {
    const { user, isAdmin } = useSharedProps();

    return (
        <AppLayout breadcrumbs={breadcrumbs}>
            <Head title="Dashboard" />
            <div className="flex flex-1 flex-col gap-4 p-4">
                {/* Banner Selamat Datang */}
                <div className="rounded-xl border border-sidebar-border/70 p-6 dark:bo
    rder-sidebar-border">
                    <h1 className="text-xl font-semibold">Selamat datang, {user.name}!</h1>
                    <p className="mt-1 flex items-center gap-2 text-sm text-muted-foreground"
                        <ShieldCheck className="size-4" />
                        Anda login sebagai {isAdmin ? 'Admin' : 'User'}.
                    </p>
                </div>

                {/* Grid Statistik */}
                <div className="grid gap-4 md:grid-cols-2">
                    {/* Card Total Pendaftar */}
                    <div className="flex flex-col justify-between rounded-xl border bord
    er-sidebar-border/70 p-6 dark:border-sidebar-border">
                        <div className="flex items-center justify-between">
                            <span className="text-sm font-medium text-muted-foreground">Total
    /span>
                            <ClipboardList className="size-5 text-muted-foreground" />
                        </div>
                        <div className="mt-4">
                            <div className="text-2xl font-bold">{totalPendaftar}</div>
                        </div>
                        <div className="mt-4">
                            <Link
                                href="/pendaftaran"
                                className="text-sm font-medium text-primary hover:underline"
                            >
                                Kelola pendaftaran &rarr;
                            </Link>
                        </div>
                    </div>

                    {/* Card Total Users (Khusus Admin) */}
                    {isAdmin && totalUsers !== null && (
                        <div className="flex flex-col justify-between rounded-xl border bo
    rder-sidebar-border/70 p-6 dark:border-sidebar-border">
                            <div className="flex items-center justify-between">
                                <span className="text-sm font-medium text-muted-foreground">T
    </span>
                                <Users className="size-5 text-muted-foreground" />
                            </div>
                            <div className="mt-4">
                                <div className="text-2xl font-bold">{totalUsers}</div>
                            </div>
                            <div className="mt-4">
                                <Link
                                    href="/admin/users"
                                    className="text-sm font-medium text-primary hover:underli
                                >
                                    Kelola user &rarr;
                                </Link>
                            </div>
                        </div>
                    )}
                </div>
            </div>
        </AppLayout>
    );
}
```

Kode 20. Halaman dashboard.tsx.

C.5 Authorization dengan Middleware Role

Middleware `EnsureUserHasRole` menerima daftar role sebagai parameter, misalnya `role:admin` atau `role:admin,user`. Jika pengguna belum login atau role-nya tidak termasuk daftar, permintaan dihentikan dengan `abort(403)` beserta pesan berbahasa Indonesia. Pemeriksaan memakai method `hasRole()` pada model `User` sehingga nilai role dibandingkan sebagai enum, bukan string bebas.

```php
<?php

namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;
use Symfony\Component\HttpFoundation\Response;

class EnsureUserHasRole
{
        /**
          * Ensure the authenticated user has one of the given roles.
          *
          * Usage: ->middleware('role:admin') or ->middleware('role:admin,user')
          *
          * @param  \Closure(\Illuminate\Http\Request): (\Symfony\Component\HttpFou
    ndation\Response)  $next
          */
        public function handle(Request $request, Closure $next, string ...$roles): Response
        {
                $user = $request->user();

                if ($user === null || ! $user->hasRole(...$roles)) {
                        abort(403, 'Anda tidak memiliki hak akses ke halaman ini.');
                }

                return $next($request);
        }
}
```

Kode 21. Middleware EnsureUserHasRole.

C.6 Modul UserManagement (CRUD Inertia)

Modul UserManagement memakai model `App\Models\User` milik aplikasi utama, sehingga modul ini tidak memiliki model sendiri. Aturan validasi saya taruh pada `StoreUserRequest` dan `UpdateUserRequest`. Pada `UpdateUserRequest`, email memakai `ignore` agar email milik user yang sedang diedit tidak dianggap duplikat, dan password bersifat nullable sehingga password lama tetap dipakai bila kolomnya dikosongkan. `Rule::when` membatasi pilihan role ketika admin mengubah akunnya sendiri.

```php
<?php
namespace Modules\UserManagement\Http\Requests;

use App\Enums\UserRole;
use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Validation\Rule;
use Illuminate\Validation\Rules\Password;

class StoreUserRequest extends FormRequest
{
        /**
          * Get the validation rules that apply to the request.
          *
          * @return array<string, array<int, mixed>>
          */
        public function rules(): array
        {
                return [
                        'name' => ['required', 'string', 'max:255'],
                        'email' => [
                                'required',
                                'string',
                                'email',
                                'max:255',
                                'unique:users,email'
                        ],
                        'role' => ['required', Rule::enum(UserRole::class)],
                        'password' => ['required', 'confirmed', Password::defaults()],
                ];
        }
        /**
          * Determine if the user is authorized to make this request.
          */
        public function authorize(): bool
        {
                return $this->user()?->isAdmin() ?? false;
        }
}
```

Kode 22. StoreUserRequest.



```php
<?php

namespace Modules\UserManagement\Http\Requests;

use App\Enums\UserRole;
use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Validation\Rule;
use Illuminate\Validation\Rules\Password;

class UpdateUserRequest extends FormRequest
{
        /**
          * Get the validation rules that apply to the request.
          *
          * An admin cannot demote their own account, so they never lock themselves out.
          *
          * @return array<string, array<int, mixed>>
          */
        public function rules(): array
        {
                $user = $this->route('user');
                $isEditingOwnAccount = $user?->is($this->user()) ?? false;

                return [
                        'name' => ['required', 'string', 'max:255'],
                        'email' => [
                                'required',
                                'string',
                                'email',
                                'max:255',
                                Rule::unique('users', 'email')->ignore($user?->id),
                        ],
                        'role' => [
                                'required',
                                Rule::enum(UserRole::class),
                                Rule::when($isEditingOwnAccount, Rule::in([UserRole::Admin->v
                        ],
                        'password' => ['nullable', 'confirmed', Password::defaults()],
                ];
        }

        /**
          * Get custom messages for validator errors.
          *
          * @return array<string, string>
          */
        public function messages(): array
        {
                return [
                        'role.in' => 'Anda tidak dapat menurunkan role akun Anda sendiri.',
                ];
        }

        /**
          * Determine if the user is authorized to make this request.
          */
        public function authorize(): bool
        {
                return $this->user()?->isAdmin() ?? false;
        }
}
```

Kode 23. UpdateUserRequest dengan aturan role untuk akun sendiri.

Controller mendukung pencarian nama atau email, filter role, dan pagination sepuluh baris per halaman. Setiap aksi mengembalikan halaman Inertia atau redirect beserta pesan flash. Method `destroy` menolak penghapusan akun sendiri dengan pesan error, sedangkan method `roleOptions()` menyiapkan pilihan role untuk select pada halaman React.

```php
<?php

namespace Modules\UserManagement\Http\Controllers;

use App\Enums\UserRole;
use App\Http\Controllers\Controller;
use App\Models\User;
use Illuminate\Database\Eloquent\Builder;
use Illuminate\Http\Request;
use Inertia\Inertia;
use Inertia\Response;

class UserManagementController extends Controller
{
        /**
          * Display a listing of users, optionally filtered by keyword and role.
          */
        public function index(Request $request): Response
        {
                $search = $request->string('search')->trim()->toString();
                $role = UserRole::tryFrom($request->string('role')->toString());

                $users = User::query()
                        ->when($search !== '', function (Builder $query) use ($search) {
                                $query->where(function (Builder $query) use ($search) {
                                        $query->where('name', 'like', "%{$search}%")
                                                ->orWhere('email', 'like', "%{$search}%");
                                });
                        })
                        ->when($role, fn (Builder $query) => $query->where('role', $role))
                        ->orderBy('name')
                        ->paginate(10)
                        ->withQueryString()
                        ->through(fn (User $user) => [
                                'id' => $user->id,
                                'name' => $user->name,
                                'email' => $user->email,
                                'role' => $user->role->value,
                                'role_label' => $user->role->label(),
                                'created_at' => $user->created_at?->diffForHumans(),
                        ]);

                return Inertia::render('UserManagement::index', [
                        'users' => $users,
                        'filters' => [
                                'search' => $search,
                                'role' => $role?->value ?? '',
                        ],
                        'roles' => $this->roleOptions(),
                ]);
        }
```

Kode 24. UserManagementController bagian 1 (index).

```
    /**
     * Store a newly created user.
     */
    public function store(StoreUserRequest $request): RedirectResponse
    {
        User::create($request->validated());

        return redirect()->route('admin.users.index')->with('success', 'User berhasil ditambahkan.');
    }

    /**
     * Show the form for editing the specified user.
     */
    public function edit(User $user): Response
    {
        return Inertia::render('UserManagement::edit', [
            'user' => [
                'id' => $user->id,
                'name' => $user->name,
                'email' => $user->email,
                'role' => $user->role->value,
            ],
            'roles' => $this->roleOptions(),
        ]);
    }

    /**
     * Update the specified user. The password is only changed when filled in.
     */
    public function update(UpdateUserRequest $request, User $user): RedirectResponse
    {
        $attributes = $request->validated();

        if (blank($attributes['password'] ?? null)) {
            unset($attributes['password']);
        }

        $user->update($attributes);

        return redirect()->route('admin.users.index')->with('success', 'User berhasil diperbarui.');
    }
```

Kode 25. UserManagementController bagian 2 (create, store, edit, update).

```
        /**
          * Remove the specified user. An admin cannot delete their own account.
          */
        public function destroy(Request $request, User $user): RedirectResponse
        {
                if ($user->is($request->user())) {
                        return redirect()->route('admin.users.index')->with('error', 'Anda tidak dapat menghapus akun Anda sendiri.');
                }

                $user->delete();

                return redirect()->route('admin.users.index')->with('success', 'User berhasil dihapus.');
        }

        /**
          * Role options for the select input on the React page.
          *
          * @return list<array{value: string, label: string}>
          */
        private function roleOptions(): array
        {
                return array_map(
                        fn (UserRole $role) => [
                                'value' => $role->value,
                                'label' => $role->label(),
                        ],
                        UserRole::cases(),
                );
        }
}
```

Kode 26. UserManagementController bagian 3 (destroy dan roleOptions).

Route modul didaftarkan dengan `Route::resource` di dalam grup middleware `auth` dan `role:admin`, memakai prefix `admin` dan awalan nama `admin.`, sehingga URL-nya menjadi `/admin/users`. Method `show` tidak dipakai karena halaman detail user tidak diperlukan.

```
<?php

use Illuminate\Support\Facades\Route;
use Modules\UserManagement\Http\Controllers\UserManagementController;

Route::middleware(['auth', 'role:admin'])
        ->prefix('admin')
        ->name('admin.')
        ->group(function () {
                Route::resource('users', UserManagementController::class)->except('show');
        });
```

Kode 27. Route modul UserManagement.

Halaman `index.tsx` menampilkan tabel user, kotak pencarian, filter role, dan pagination. Tombol Hapus tidak ditampilkan pada baris akun yang sedang login, dan penghapusan

memakai `ConfirmDialog` sehingga request DELETE baru dikirim setelah pengguna menekan Ya, hapus. Halaman `create` dan `edit` memakai satu komponen form bersama, yaitu `user-form.tsx`, dengan `useForm` milik Inertia.



Kode 28. Halaman index.tsx modul UserManagement.



```tsx
import { Link } from '@inertiajs/react';
import type { FormEvent } from 'react';
import InputError from '@/components/input-error';
import { Button } from '@/components/ui/button';
import { Input } from '@/components/ui/input';
import { Label } from '@/components/ui/label';

export type UserFormData = {
        name: string;
        email: string;
        role: string;
        password: string;
        password_confirmation: string;
};

type UserFormProps = {
        data: UserFormData;
        errors: Partial<Record<keyof UserFormData, string>>;
        processing: boolean;
        roles: { value: string; label: string }[];
        isEdit?: boolean;
        submitLabel: string;
        setData: (key: keyof UserFormData, value: string) => void;
        onSubmit: (event: FormEvent) => void;
};

export function UserForm({
        data,
        errors,
        processing,
        roles,
        isEdit = false,
        submitLabel,
        setData,
        onSubmit,
}: UserFormProps) {
        return (
                <form onSubmit={onSubmit} noValidate className="max-w-xl space-y-5 rou
    nded-xl border p-6">
                        <div className="grid gap-2">
                                <Label htmlFor="name">
                                        Nama <span className="text-red-500">*</span>
                                </Label>
                                <Input
                                        id="name"
                                        value={data.name}
                                        onChange={(event) => setData('name', event.target.val
                                        autoFocus
                                />
                                <InputError message={errors.name} />
                        </div>

                        <div className="grid gap-2">
                                <Label htmlFor="email">
                                        Email <span className="text-red-500">*</span>
                                </Label>
                                <Input
                                        id="email"
                                        type="email"
                                        value={data.email}
                                        onChange={(event) => setData('email', event.target.va
                                />
                                <InputError message={errors.email} />
                        </div>

                        <div className="grid gap-2">
                                <Label htmlFor="role">
                                        Role <span className="text-red-500">*</span>
                                </Label>
                                <select
                                        id="role"
                                        value={data.role}
                                        onChange={(event) => setData('role', event.target.val
                                        className="h-9 rounded-md border border-input bg-tran
    nt px-3 text-sm shadow-xs"
                                >
                                        {roles.map((role) => (
                                                <option key={role.value} value={role.value}>
                                                        {role.label}
                                                </option>
                                        ))}
                                </select>
                                <InputError message={errors.role} />
                        </div>

                        <div className="grid gap-2">
                                <Label htmlFor="password">
                                        Password {!isEdit && <span className="text-red-500">*
                                </Label>
                                <Input
                                        id="password"
                                        type="password"
                                        value={data.password}
                                        onChange={(event) => setData('password', event.target
                                />
                                {isEdit && (
                                        <p className="text-xs text-muted-foreground">
                                                Kosongkan jika tidak ingin mengganti password
                                        </p>
                                )}
                                <InputError message={errors.password} />
                        </div>

                        <div className="grid gap-2">
                                <Label htmlFor="password_confirmation">Konfirmasi Password</L
                                <Input
                                        id="password_confirmation"
                                        type="password"
                                        value={data.password_confirmation}
                                        onChange={(event) => setData('password_confirmation',
    value)}
                                />
                        </div>

                        <div className="flex gap-2">
                                <Button type="submit" disabled={processing}>
                                        {processing ? 'Menyimpan...' : submitLabel}
                                </Button>
                                <Button asChild variant="secondary">
                                        <Link href="/admin/users">Batal</Link>
                                </Button>
                        </div>
                </form>
        );
}
```

Kode 29. Komponen user-form.tsx.

```tsx
import { Head, useForm } from '@inertiajs/react';
import type { FormEvent } from 'react';
import AppLayout from '@/layouts/app-layout';
import type { BreadcrumbItem } from '@/types';
import { UserForm } from '../components/user-form';
import type { UserFormData } from '../components/user-form';
type CreateProps = {
        roles: { value: string; label: string }[];
};
const breadcrumbs: BreadcrumbItem[] = [
        { title: 'Kelola User', href: '/admin/users' },
        { title: 'Tambah User', href: '/admin/users/create' },
];
export default function Create({ roles }: CreateProps) {
        const { data, setData, post, processing, errors } = useForm<UserFormData>({
                name: '',
                email: '',
                role: 'user',
                password: '',
                password_confirmation: '',
        });
        const submit = (event: FormEvent) => {
                event.preventDefault();
                post('/admin/users');
        };
        return (
                <AppLayout breadcrumbs={breadcrumbs}>
                        <Head title="Tambah User" />
                        <div className="flex flex-1 flex-col gap-4 p-4">
                                <div>
                                        <h1 className="text-xl font-semibold">Tambah User</h1
                                        <p className="text-sm text-muted-foreground">Isi data
                                                beserta role-nya.</p>
                                </div>
                                <UserForm
                                        data={data}
                                        errors={errors}
                                        processing={processing}
                                        roles={roles}
                                        submitLabel="Simpan"
                                        setData={setData}
                                        onSubmit={submit}

                                />
                        </div>
                </AppLayout>
        );
}
```

Kode 30. Halaman create.tsx modul UserManagement.

```tsx
import { Head, useForm } from '@inertiajs/react';
import type { FormEvent } from 'react';
import AppLayout from '@/layouts/app-layout';
import type { BreadcrumbItem } from '@/types';
import { UserForm } from '../components/user-form';
import type { UserFormData } from '../components/user-form';
type EditProps = {
        user: { id: number; name: string; email: string; role: string };
        roles: { value: string; label: string }[];
};
export default function Edit({ user, roles }: EditProps) {
        const breadcrumbs: BreadcrumbItem[] = [
                { title: 'Kelola User', href: '/admin/users' },
                { title: 'Edit User', href: `/admin/users/${user.id}/edit` },
        ];
        const { data, setData, put, processing, errors } = useForm<UserFormData>({
                name: user.name,
                email: user.email,
                role: user.role,
                password: '',
                password_confirmation: '',
        });
        const submit = (event: FormEvent) => {
                event.preventDefault();
                put(`/admin/users/${user.id}`);
        };
        return (
                <AppLayout breadcrumbs={breadcrumbs}>
                        <Head title="Edit User" />
                        <div className="flex flex-1 flex-col gap-4 p-4">
                                <div>
                                        <h1 className="text-xl font-semibold">Edit User</h1>
                                        <p className="text-sm text-muted-foreground">Perbarui
                                                {user.name}.</p>
                                </div>
                                <UserForm
                                        data={data}
                                        errors={errors}
                                        processing={processing}
                                        roles={roles}
                                        isEdit
                                        submitLabel="Simpan Perubahan"
                                        setData={setData}
                                        onSubmit={submit}

                                />
                        </div>
                </AppLayout>
        );
}
```

Kode 31. Halaman edit.tsx modul UserManagement.

C.7 Modul Pendaftaran (CRUD AJAX)

Modul Pendaftaran menyimpan data calon mahasiswa beserta satu berkas pendukung. Migration pertama membuat tabel `daftars`, sedangkan migration kedua menambahkan kolom `berkas` dan `nama_berkas`. Pemisahan ini saya lakukan karena kolom berkas ditambahkan setelah tabel awal terbentuk, sehingga riwayat perubahan skema tetap terbaca. Kolom `no_hp` dibuat sepanjang 20 karakter agar angka 0 di depan tidak hilang.

```php
<?php
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;
return new class extends Migration {
        /**
          * Run the migrations.
          */
        public function up(): void
        {
                Schema::create('daftars', function (Blueprint $table) {
                        $table->id();
                        $table->string('nama');
                        $table->string('asal_sekolah');
                        $table->string('prodi_tujuan');
                        $table->string('email')->unique();
                        $table->string('no_hp', 20);
                        $table->timestamps();
                });
        }
        /**
          * Reverse the migrations.
          */
        public function down(): void
        {
                Schema::dropIfExists('daftars');
        }
};
```

Kode 32. Migration tabel daftars.

```php
<?php
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;
return new class extends Migration {
        /**
          * Run the migrations.
          */
        public function up(): void
        {
                Schema::table('daftars', function (Blueprint $table) {
                        $table->string('berkas')->nullable()->after('no_hp');
                        $table->string('nama_berkas')->nullable()->after('berkas');
                });
        }
        /**
          * Reverse the migrations.
          */
        public function down(): void
        {
                Schema::table('daftars', function (Blueprint $table) {
                        $table->dropColumn(['berkas', 'nama_berkas']);
                });
        }
};
```

Kode 33. Migration penambah kolom berkas dan nama_berkas.

Factory dipakai untuk data uji. Model `Daftar` mengisi `$fillable` dan menambahkan accessor `berkas_url` yang diappend ke JSON. Accessor itu mengubah path penyimpanan menjadi URL lengkap lewat `asset('storage/...')`, sehingga React tidak perlu menyusun alamat berkas sendiri. Factory milik modul didaftarkan melalui `newFactory()` karena letaknya di luar folder `factories` bawaan Laravel.

```php
<?php
namespace Modules\Pendaftaran\Models;
use Illuminate\Database\Eloquent\Casts\Attribute;
use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Modules\Pendaftaran\Database\Factories\DaftarFactory;
class Daftar extends Model
{
        /** @use HasFactory<DaftarFactory> */
        use HasFactory;
        /**
          * The attributes that are mass assignable.
          *
          * @var list<string>
          */
        protected $fillable = [
                'nama',
                'asal_sekolah',
                'prodi_tujuan',
                'email',
                'no_hp',
                'berkas',
                'nama_berkas',
        ];
        /**
          * The accessors to append to the model's array and JSON form.
          *
          * @var list<string>
          */
        protected $appends = [
                'berkas_url',
        ];
        /**
          * Public URL of the uploaded file, or null when there is no file.
          */
        protected function berkasUrl(): Attribute
        {
                return Attribute::get(fn(): ?string => $this->berkas ?
                        asset('storage/' . $this->berkas) : null);
        }
        protected static function newFactory(): DaftarFactory
        {
                return DaftarFactory::new();
        }
}
```

Kode 34. Model Daftar dengan accessor berkas_url.

```php
<?php
namespace Modules\Pendaftaran\Database\Factories;
use Illuminate\Database\Eloquent\Factories\Factory;
use Modules\Pendaftaran\Models\Daftar;
/**
  * @extends Factory<Daftar>
  */
class DaftarFactory extends Factory
{
        /**
          * The name of the factory's corresponding model.
          */
        protected $model = Daftar::class;
        /**
          * Define the model's default state.
          *
          * @return array<string, mixed>
          */
        public function definition(): array
        {
                return [
                        'nama' => fake()->name(),
                        'asal_sekolah' => 'SMA ' . fake()->city(),
                        'prodi_tujuan' => fake()->randomElement([
                                'Informatika',
                                'Sistem
Informasi',
                                'Teknik Elektro'
                        ]),
                        'email' => fake()->unique()->safeEmail(),
                        'no_hp' => fake()->numerify('08##########'),
                ];
        }
}
```

Kode 35. DaftarFactory untuk data uji.

`DaftarRequest` memakai aturan berkas yang berbeda untuk tambah dan ubah. Saat tambah, berkas wajib diunggah, sedangkan saat edit boleh dikosongkan karena berkas lama tetap dipakai. Format dibatasi jpg, jpeg, png, atau pdf dengan ukuran maksimal 2 MB, dan pesan galat untuk no HP serta berkas saya tulis dalam Bahasa Indonesia lewat method `messages()`.



```php
<?php
namespace Modules\Pendaftaran\Http\Requests;
use Illuminate\Foundation\Http\FormRequest;
use Illuminate\Validation\Rule;
class DaftarRequest extends FormRequest
{
        /**
        * Get the validation rules that apply to the request.
        *
        * The file is required when creating, and optional when updating (keep the old
        file).
        *
        * @return array<string, array<int, mixed>>
        */
        public function rules(): array
        {
                $daftar = $this->route('daftar');
                return [
                        'nama' => ['required', 'string', 'max:255'],
                        'asal_sekolah' => ['required', 'string', 'max:255'],
                        'prodi_tujuan' => ['required', 'string', 'max:255'],
                        'email' => [
                                'required',
                                'email',
                                'max:255',
                                Rule::unique(
                                        'daftars',
                                        'email'
                                )->ignore($daftar?->id)
                        ],
                        'no_hp' => ['required', 'string', 'regex:/^[0-9+]{8,20}$/'],
                        'berkas' => [
                                $daftar ? 'nullable' : 'required',
                                'file',
                                'mimes:jpg,jpeg,png,pdf',
                                'max:2048'
                        ],
                ];
        }
        /**
          * Get custom messages for validator errors.
          *
          * @return array<string, string>
          */
        public function messages(): array
        {
                return [
                        'no_hp.regex' => 'No HP hanya boleh berisi angka (8-20 digit).',
                        'berkas.required' => 'Berkas wajib diunggah.',
                        'berkas.mimes' => 'Berkas harus berformat jpg, jpeg, png, atau pdf.',
                        'berkas.max' => 'Ukuran berkas maksimal 2 MB.',
                ];
        }
        /**
          * Determine if the user is authorized to make this request.
          */
        public function authorize(): bool
        {
                return true;
        }
}
```

Kode 36. DaftarRequest dengan validasi berkas.

Controller hanya memakai satu method untuk memuat halaman, karena isi tabel diambil terpisah lewat `getData()`. Method `store`, `update`, dan `destroy` mengembalikan JSON berisi pesan berhasil. Berkas disimpan dengan `store()` ke `storage/app/public`, sedangkan database hanya menyimpan path dan nama asli berkas. Saat berkas diganti atau data dihapus, berkas lama ikut dihapus dari storage.



```php
<?php
namespace Modules\Pendaftaran\Http\Controllers;
use App\Http\Controllers\Controller;
use Illuminate\Http\JsonResponse;
use Illuminate\Support\Facades\Storage;
use Inertia\Inertia;
use Inertia\Response;
use Modules\Pendaftaran\Http\Requests\DaftarRequest;
use Modules\Pendaftaran\Models\Daftar;
class PendaftaranController extends Controller
{
        /**
          * Display the React page. Data is loaded via AJAX.
          */
        public function index(): Response
        {
                return Inertia::render('Pendaftaran::index');
        }
        /**
          * Return all registrations as JSON.
          */
        public function getData(): JsonResponse
        {
                return response()->json(Daftar::latest()->get());
        }
        /**
          * Store a newly created registration together with its uploaded file.
          */
        public function store(DaftarRequest $request): JsonResponse
        {
                $data = $request->safe()->except('berkas');
                $file = $request->file('berkas');
                $data['berkas'] = $file->store('berkas_pendaftaran', 'public');
                $data['nama_berkas'] = $file->getClientOriginalName();
                Daftar::create($data);
                return response()->json(['success' => 'Data berhasil disimpan.']);
        }
        /**
          * Return a single registration for the edit form.
          */
        public function edit(Daftar $daftar): JsonResponse
        {
                return response()->json($daftar);
        }
        /**
        * Update the specified registration. The old file is replaced only when a new
        one is uploaded.
        */
        public function update(DaftarRequest $request, Daftar $daftar): JsonResponse
        {
                $data = $request->safe()->except('berkas');
                if ($request->hasFile('berkas')) {
                        if ($daftar->berkas) {
                                Storage::disk('public')->delete($daftar->berkas);
                        }
                        $file = $request->file('berkas');
                        $data['berkas'] = $file->store('berkas_pendaftaran', 'public');
                        $data['nama_berkas'] = $file->getClientOriginalName();
                }
                $daftar->update($data);
                return response()->json(['success' => 'Data berhasil diperbarui.']);
        }
        /**
          * Remove the specified registration and its uploaded file.
          */
        public function destroy(Daftar $daftar): JsonResponse
        {
                if ($daftar->berkas) {
                        Storage::disk('public')->delete($daftar->berkas);
                }
                $daftar->delete();
                return response()->json(['success' => 'Data berhasil dihapus.']);
        }
}
```

Kode 37. PendaftaranController (index, getData, store, edit, update, destroy).

Route modul dilindungi middleware `auth` dan `role:admin,user`. Route `pendaftaran/data` saya tulis sebelum route `pendaftaran/{daftar}` agar kata `data` tidak dianggap sebagai id data.

```php
<?php
use Illuminate\Support\Facades\Route;
use Modules\Pendaftaran\Http\Controllers\PendaftaranController;

Route::middleware(['auth', 'role:admin,user'])->group(function () {
Route::get('pendaftaran', [PendaftaranController::class, 'index'])->name('pend
    aftaran.index');
Route::get('pendaftaran/data', [PendaftaranController::class, 'getData'])->nam
    e('pendaftaran.data');
Route::post('pendaftaran', [PendaftaranController::class, 'store'])->name('pen
    daftaran.store');
Route::get('pendaftaran/{daftar}/edit', [PendaftaranController::class, 'edit']
    )->name('pendaftaran.edit');
Route::put('pendaftaran/{daftar}', [PendaftaranController::class, 'update'])->
    name('pendaftaran.update');
Route::delete('pendaftaran/{daftar}', [PendaftaranController::class, 'destroy'
    ])->name('pendaftaran.destroy');
});
```

Kode 38. Route modul Pendaftaran.

Untuk mengirim request AJAX saya membuat helper `http.ts` di `resources/js/lib`. Helper ini membaca token CSRF dari cookie `XSRF-TOKEN` lalu mengirimnya sebagai header `X-XSRF-TOKEN`, dan menambahkan `Accept: application/json` agar Laravel mengembalikan error validasi sebagai JSON berkode 422. Jika respons bernilai gagal, helper melempar `HttpError` yang membawa status dan isi JSON.

```ts
export type ValidationErrors = Record<string, string[]>;
type ErrorBody = { message?: string; errors?: ValidationErrors } | null;
export class HttpError extends Error {
        status: number;
        data: ErrorBody;
        constructor(status: number, data: ErrorBody) {
                super(data?.message ?? `Request gagal dengan status ${status}`);
                this.status = status;
                this.data = data;
        }
}
// Membaca token CSRF dari cookie XSRF-TOKEN yang dibuat oleh Laravel
function getXsrfToken(): string {
        const cookie = document.cookie.split('; ').find((row) => row.startsWith('XSRF-TOKEN='
    );

        return cookie ? decodeURIComponent(cookie.substring('XSRF-TOKEN='.length)) : '';
}
type RequestOptions = {
        method?: 'GET' | 'POST' | 'PUT' | 'DELETE';
        body?: FormData;
};
export async function http<T>(url: string, { method = 'GET', body }: RequestOptions =    {
    }): Promise<T> {
        const response = await fetch(url, {
                method,
                body,
                credentials: 'same-origin',
                headers: {
                        Accept: 'application/json',
                        'X-Requested-With': 'XMLHttpRequest',
                        'X-XSRF-TOKEN': getXsrfToken(),
                },
        });
        const data = await response.json().catch(() => null);
        if (!response.ok) {
                throw new HttpError(response.status, data);
        }
        return data as T;
}
```

Kode 39. Helper http.ts untuk request AJAX.

Halaman `index.tsx` memuat data sekali lewat `GET /pendaftaran/data`, lalu seluruh operasi tambah, ubah, dan hapus berjalan tanpa memuat ulang halaman. Form tambah dan edit memakai satu Dialog yang sama; state `editing` bernilai null saat menambah dan berisi data pendaftar saat edit. Seluruh form dikirim sebagai `FormData` karena data berisi file, dan saat edit ditambahkan `_method` bernilai `PUT` karena PHP hanya membaca file pada request POST.

```tsx
import { Head } from '@inertiajs/react';
import { FileImage, FileText, LoaderCircle, Pencil, Plus, Search, Trash2 } from
    'lucide-react';
import { useEffect, useMemo, useState } from 'react';
import type { ChangeEvent, FormEvent } from 'react';
import { AlertMessage } from '@/components/alert-message';
import { ConfirmDialog } from '@/components/confirm-dialog';
import InputError from '@/components/input-error';
import { Dialog, DialogContent, DialogDescription, DialogFooter,
                DialogHeader, DialogTitle } from '@/components/ui/dialog';
import { http, HttpError } from '@/lib/http';
import type { ValidationErrors } from '@/lib/http';

type Daftar = {
        id: number;
        nama: string;
        asal_sekolah: string;
        prodi_tujuan: string;
        email: string;
        no_hp: string;
        berkas: string | null;
        nama_berkas: string | null;
        berkas_url: string | null;
};

type FormFields = Pick<Daftar, 'nama' | 'asal_sekolah' | 'prodi_tujuan' | 'email' |
    'no_hp'>;
type Message = { type: 'success' | 'error'; text: string } | null;

const emptyForm: FormFields = {
        nama: '',
        asal_sekolah: '',
        prodi_tujuan: '',
        email: '',
        no_hp: '',
};

const fields: { name: keyof FormFields; label: string; type?: string }[] = [
        { name: 'nama', label: 'Nama' },
        { name: 'asal_sekolah', label: 'Asal Sekolah' },
        { name: 'prodi_tujuan', label: 'Prodi Tujuan' },
        { name: 'email', label: 'Email', type: 'email' },
        { name: 'no_hp', label: 'No HP' },
];

const urls = {
        data: '/pendaftaran/data',
        store: '/pendaftaran',
        edit: (id: number) => `/pendaftaran/${id}/edit`,
        update: (id: number) => `/pendaftaran/${id}`,
        destroy: (id: number) => `/pendaftaran/${id}`,
};

const breadcrumbs: BreadcrumbItem[] = [{ title: 'Pendaftaran', href: '/pendaftaran' }];

// Tautan berkas yang dibuka di tab baru, ikon dipilih berdasarkan ekstensi
function FileLink({ url, name }: { url: string | null; name: string | null }) {
        if (!url) {
                return <span className="text-muted-foreground">-</span>;
        }

        const Icon = name?.toLowerCase().endsWith('.pdf') ? FileText : FileImage;

        return (
                <a
                        href={url}
                        target="_blank"
                        rel="noopener noreferrer"
                        className="inline-flex items-center gap-1 text-primary underline-o
    ffset-4 hover:underline"
                >
                        <Icon className="size-4" /> {name ?? 'Lihat berkas'}
                </a>
        );
}
```

Kode 40. index.tsx modul Pendaftaran bagian 1 (tipe, url, dan FileLink).

```tsx
export default function Index() {
        const [records, setRecords] = useState<Daftar[]>([]);
        const [loading, setLoading] = useState(true);
        const [keyword, setKeyword] = useState('');
        const [message, setMessage] = useState<Message>(null);
        const [dialogOpen, setDialogOpen] = useState(false);
        const [editing, setEditing] = useState<Daftar | null>(null);
        const [form, setForm] = useState<FormFields>(emptyForm);
        const [file, setFile] = useState<File | null>(null);
        const [fileInputKey, setFileInputKey] = useState(0);
        const [errors, setErrors] = useState<ValidationErrors>({});
        const [saving, setSaving] = useState(false);
        const [deleting, setDeleting] = useState<Daftar | null>(null);
        const [processingDelete, setProcessingDelete] = useState(false);

        const showError = (error: unknown, fallback: string) => {
                const text = error instanceof HttpError ? (error.data?.message ?? fallback) :
    fallback;
                setMessage({ type: 'error', text });
        };

        // Mengambil seluruh data pendaftar dari server (GET /pendaftaran/data)
        const loadData = async () => {
                setLoading(true);

                try {
                        setRecords(await http<Daftar[]>(urls.data));
                } catch (error) {
                        showError(error, 'Gagal memuat data pendaftar.');
                } finally {
                        setLoading(false);
                }
        };

        useEffect(() => {
                loadData();
        }, []);

        // Pencarian dilakukan di browser tanpa request baru ke server
        const filtered = useMemo(() => {
                const q = keyword.trim().toLowerCase();

                if (q === '') {
                        return records;
                }

                return records.filter((item) =>
                        [item.nama, item.asal_sekolah, item.prodi_tujuan, item.email, item.no
    ((value) =>
                                value.toLowerCase().includes(q),
                        ),
                );
        }, [records, keyword]);
```

Kode 41. index.tsx modul Pendaftaran bagian 2 (state, memuat data, pencarian).



```
// Menyimpan data (tambah dan edit) menggunakan FormData
const handleSubmit = async (event: FormEvent) => {
        event.preventDefault();
        setSaving(true);
        setErrors({});

        const formData = new FormData();
        Object.entries(form).forEach(([key, value]) => formData.append(key, value));

        if (file) {
                formData.append('berkas', file);
        }

        if (editing) {
                formData.append('_method', 'PUT');
        }

        try {
                const result = await http<{ success: string }>(
                        editing ? urls.update(editing.id) : urls.store,
                        {
                                method: 'POST',
                                body: formData,
                        },
                );
                setDialogOpen(false);
                setMessage({ type: 'success', text: result.success });
                await loadData();
        } catch (error) {
                if (error instanceof HttpError && error.status === 422) {
                        setErrors(error.data?.errors ?? {});
                } else {
                        showError(error, 'Terjadi kesalahan, silakan coba lagi.');
                }
        } finally {
                setSaving(false);
        }
};

// Menghapus data (DELETE /pendaftaran/{id}) setelah dikonfirmasi
const handleDelete = async () => {
        if (!deleting) {
                return;
        }

        setProcessingDelete(true);

        try {
                const result = await http<{ success: string }>(urls.destroy(deleting.
                        method: 'DELETE',
                });
                setMessage({ type: 'success', text: result.success });
                await loadData();
        } catch (error) {
                showError(error, 'Data gagal dihapus.');
        } finally {
                setProcessingDelete(false);
                setDeleting(null);
        }
};
```

Kode 42. index.tsx modul Pendaftaran bagian 3 (simpan dan hapus dengan AJAX).

```
        return (
                <AppLayout breadcrumbs={breadcrumbs}>
                        <Head title="Pendaftaran" />
                        <div className="flex flex-1 flex-col gap-4 p-4">
                                <div className="flex flex-wrap items-center justify-between g
                                        <div>
                                                <h1 className="flex items-center gap-2 text-x
                                                        Data Pendaftar <Badge variant="second
    dge>
                                                </h1>
                                                <p className="text-sm text-muted-foreground">
                                                        Seluruh operasi pada halaman ini berj
    memuat
                                                        ulang halaman.
                                                </p>
                                        </div>

                                        <Button onClick={openCreate}>
                                                <Plus /> Tambah Pendaftar
                                        </Button>
                                </div>

                                <AlertMessage
                                        type={message?.type ?? 'success'}
                                        message={message?.text}
                                        onClose={() => setMessage(null)}
                                />

                                <div className="relative max-w-sm">
                                        <Search className="absolute top-1/2 left-3 size-4 -tr
    te-y-1/2 text-muted-foreground" />
                                        <Input
                                                value={keyword}
                                                onChange={(event) => setKeyword(event.target.
                                                placeholder="Cari nama, sekolah, prodi, email
                                                className="pl-9"
                                        />
                                </div>
/* ... markup tombol Edit/Hapus ... */
                </AppLayout>
        );
}
```

Kode 43. index.tsx modul Pendaftaran bagian 4 (markup tabel).



```
                        <Dialog
                                open={dialogOpen}
                                onOpenChange={(open) => !saving && setDialogOpen(open)}
                        >
                                <DialogContent className="sm:max-w-lg">
                                        <form onSubmit={handleSubmit} noValidate className="s
                                                <DialogHeader>
                                                        <DialogTitle>
                                                                {editing ? 'Edit Pendaftar' :
                                                        </DialogTitle>
                                                        <DialogDescription>
                                                                Lengkapi data calon mahasiswa
                                                        </DialogDescription>
                                                </DialogHeader>

                                                {fields.map((field) => (
                                                        <div key={field.name} className="grid
                                                                <Label htmlFor={field.name}>
                                                                        {field.label} <span c
                                                                </Label>
                                                                <Input
                                                                        id={field.name}
                                                                        name={field.name}
                                                                        type={field.type ?? '
                                                                        value={form[field.nam
                                                                        onChange={handleChang
                                                                        aria-invalid={!!error
                                                                />
                                                                <InputError message={errors[f
                                                        </div>
                                                ))}

                                                <div className="grid gap-2">
                                                        <Label htmlFor="berkas">
                                                                Berkas Pendukung{' '}
                                                                {!editing && <span className=
                                                        </Label>
                                                        <Input
                                                                key={fileInputKey}
                                                                id="berkas"
                                                                type="file"
                                                                accept=".jpg,.jpeg,.png,.pdf"
                                                                onChange={(event) => setFile(
    null)}
                                                                aria-invalid={!!errors.berkas
                                                        />
                                                        <p className="text-xs text-muted-fore
                                                                Format jpg, jpeg, png, atau p
                                                                {editing && ' Kosongkan jika '
    anti berkas.'}
                                                        </p>

                                                        {editing?.berkas_url && (
                                                                <p className="text-xs">
                                                                        Berkas saat ini:{' '}
                                                                        <FileLink url={editin
    nama_berkas} />
                                                                </p>
                                                        )}

                                                        <InputError message={errors.berkas?.[
                                                </div>

                                                <DialogFooter className="gap-2">
                                                        <Button
                                                                type="button"
                                                                variant="secondary"
                                                                onClick={() => setDialogOpen(
                                                                disabled={saving}
                                                        >
                                                                Batal
                                                        </Button>
                                                        <Button type="submit" disabled={savin
                                                                {saving && <LoaderCircle clas
                                                                {saving ? 'Menyimpan...' : 'S
                                                        </Button>
                                                </DialogFooter>
                                        </form>
                                </DialogContent>
                        </Dialog>
                        </Dialog>
```

Kode 44. index.tsx modul Pendaftaran bagian 5 (dialog form).

```
                                                <ConfirmDialog
                                open={deleting !== null}
                                description={`Yakin ingin menghapus data ${deleting?.nama}
        beserta berkasnya?`}
                                processing={processingDelete}
                                onConfirm={handleDelete}
                                onOpenChange={(open) => !open && setDeleting(null)}
                            />
                        </AppLayout>
        );
}
}
```

Kode 45. index.tsx modul Pendaftaran bagian 6 (dialog lanjutan dan konfirmasi hapus).

C.8 Menjalankan dan Menguji Aplikasi

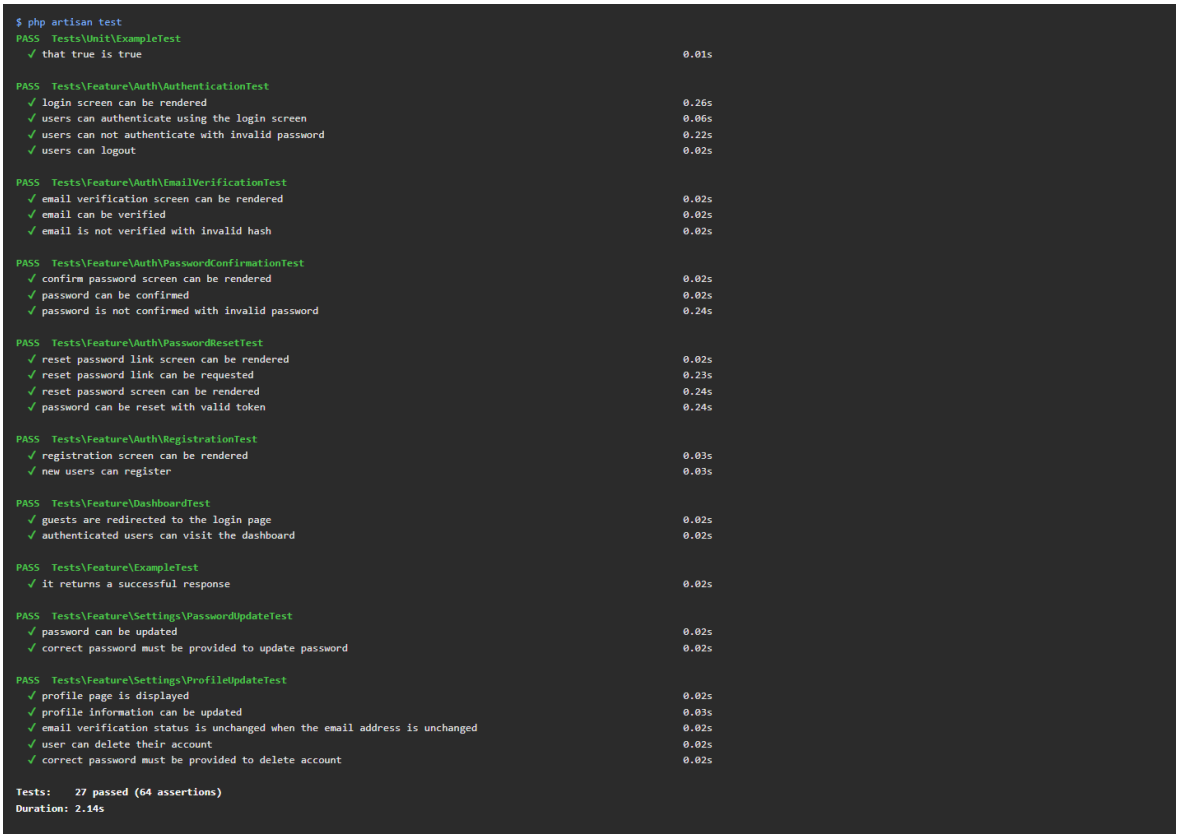
Aplikasi dijalankan dengan `php artisan serve` pada port 8010, sedangkan aset React dilayani dari hasil `npm run build`. Database diisi ulang dengan `php artisan migrate:fresh --seed` sehingga hanya berisi dua akun seeder. Halaman login, register, dan dashboard berasal dari starter kit, dan hanya halaman kedua modul yang ditulis di dalam folder `Modules`.



Gambar 1. Server aplikasi berjalan pada http://127.0.0.1:8010.

C.9 Pengujian Otomatis

Selain diuji melalui browser, fitur yang ada diuji secara otomatis memakai Pest. React Starter Kit sudah menyediakan pengujian authentication bawaan, sehingga saya cukup memastikan seluruhnya berjalan pada database kosong berkat `RefreshDatabase` yang diaktifkan pada `tests/Pest.php`. Pengujian mencakup render halaman login dan register, proses login dengan password salah, logout, reset password, pembaruan password, pembaruan profil, akses dashboard, serta pengujian unit contoh. Hasil terakhir adalah 27 test lulus dengan 64 assertion.



Gambar 2. Hasil pengujian otomatis dengan php artisan test.

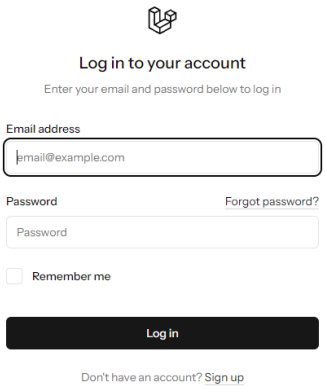
Pengujian modul sendiri belum saya tambahkan pada folder `tests`. Akibatnya, perilaku dua modul saya buktikan melalui browser dan pemeriksaan langsung pada tab Network, route list, serta isi database.

D. Hasil dan Dokumentasi Program

Berikut tangkapan layar dari aplikasi yang sudah dijalankan pada alamat `http://127.0.0.1:8010`. Aplikasi berjalan dengan `php artisan serve` dan aset React hasil `npm run build`, sedangkan database diisi ulang dengan `php artisan migrate:fresh --seed`.

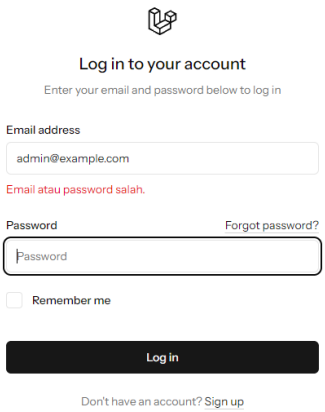
D.1 Autentikasi

Saat `/pendaftaran` dibuka tanpa login, middleware `auth` mengarahkan pengguna ke halaman login. Alamat di bilah browser berubah menjadi `/login`, sehingga halaman modul tidak dapat dilihat tamu.



Gambar 3. Halaman login saat /pendaftaran dibuka tanpa login.

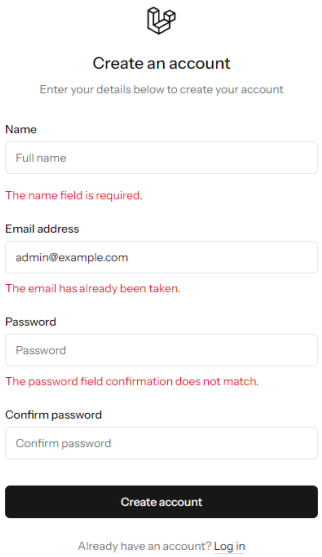
Login dengan password salah tidak mengalihkan pengguna, tetapi menampilkan pesan Email atau password salah tepat di bawah kolom email.



Gambar 4. Pesan galat saat password salah.

Halaman register menolak data yang tidak lengkap atau tidak valid. Pesan galat muncul di bawah setiap input yang salah, dan form tetap terbuka agar pengguna dapat memperbaikinya. Pada halaman ini nama dikosongkan, email memakai alamat yang sudah terdaftar, dan

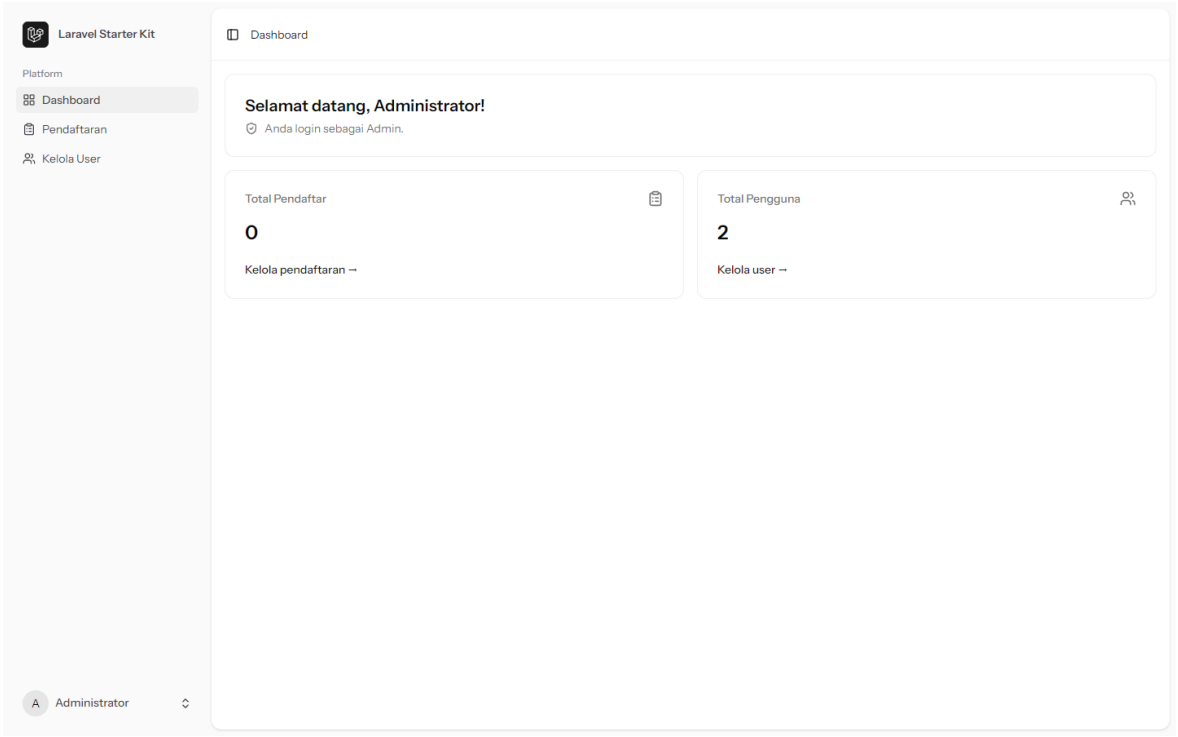
konfirmasi password sengaja dibedakan dari password. Ketiganya ditolak sekaligus dengan pesan masing-masing.



Gambar 5. Galat validasi pada form register.

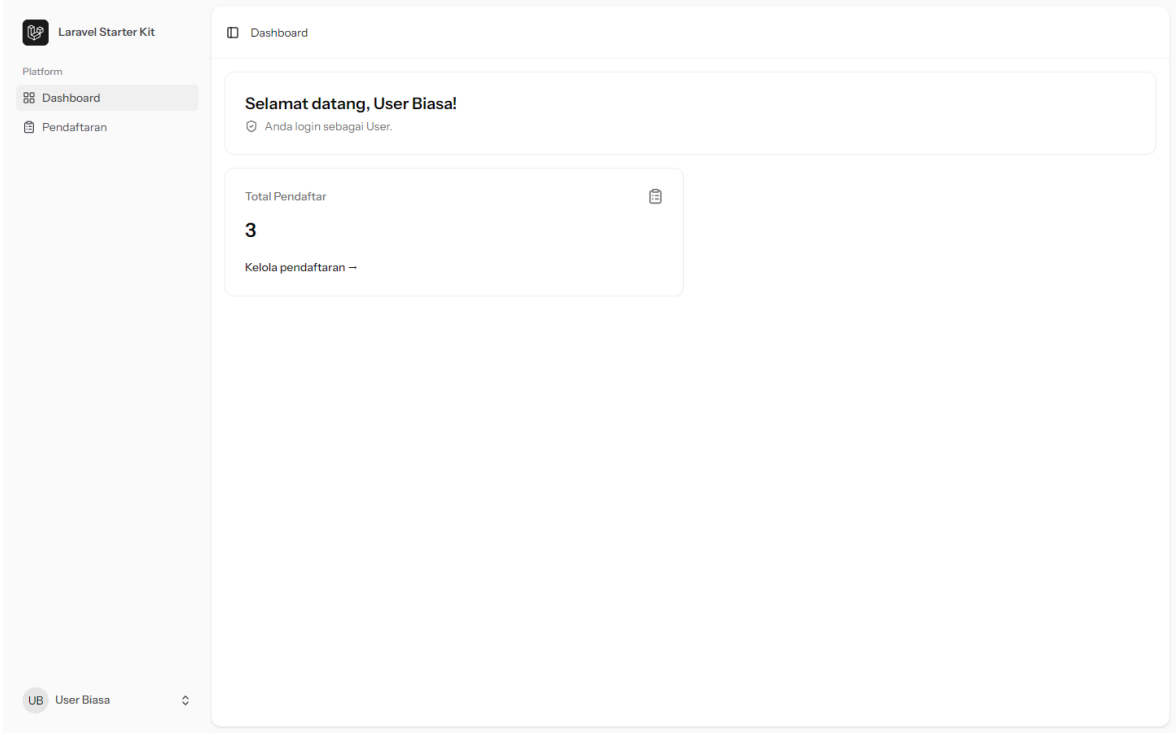
D.2 Dashboard dan Hak Akses

Dashboard akun admin menampilkan dua kartu, yaitu Total Pendaftar dan Total Pengguna, serta menu Kelola User pada sidebar. Banner menyebut bahwa pengguna login sebagai Admin.



Gambar 6. Dashboard akun admin dengan dua kartu dan menu Kelola User.

Setelah login sebagai user, dashboard hanya menampilkan satu kartu, yaitu Total Pendaftar. Menu Kelola User tidak muncul karena `isAdmin` bernilai false, dan kartu Total Pengguna tidak dirender karena props `totalUsers` bernilai null.



Gambar 7. Dashboard akun user dengan satu kartu.

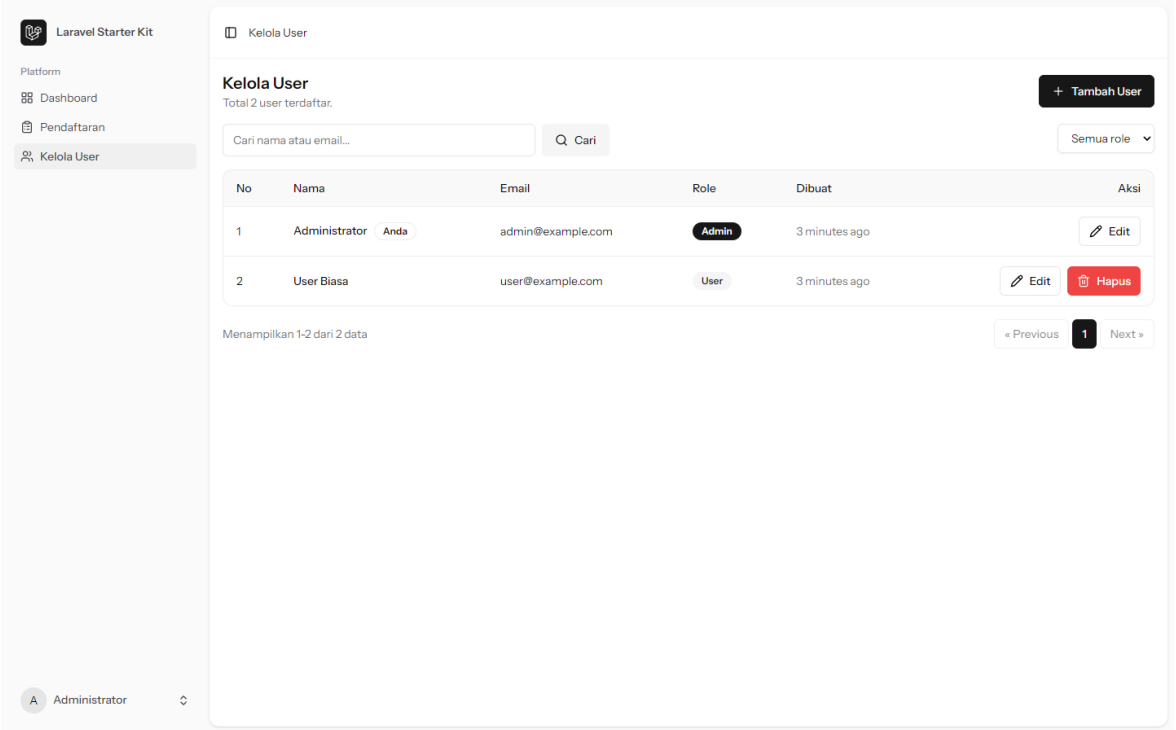
User biasa yang membuka alamat `/admin/users` secara langsung tetap ditolak oleh middleware `role` dengan kode 403 beserta pesan bahwa pengguna tidak memiliki hak akses. Ini membuktikan bahwa penyembunyian menu di React hanya mengatur tampilan, sedangkan penjaga sebenarnya berada di server.



Gambar 8. Error 403 saat user membuka halaman khusus admin.

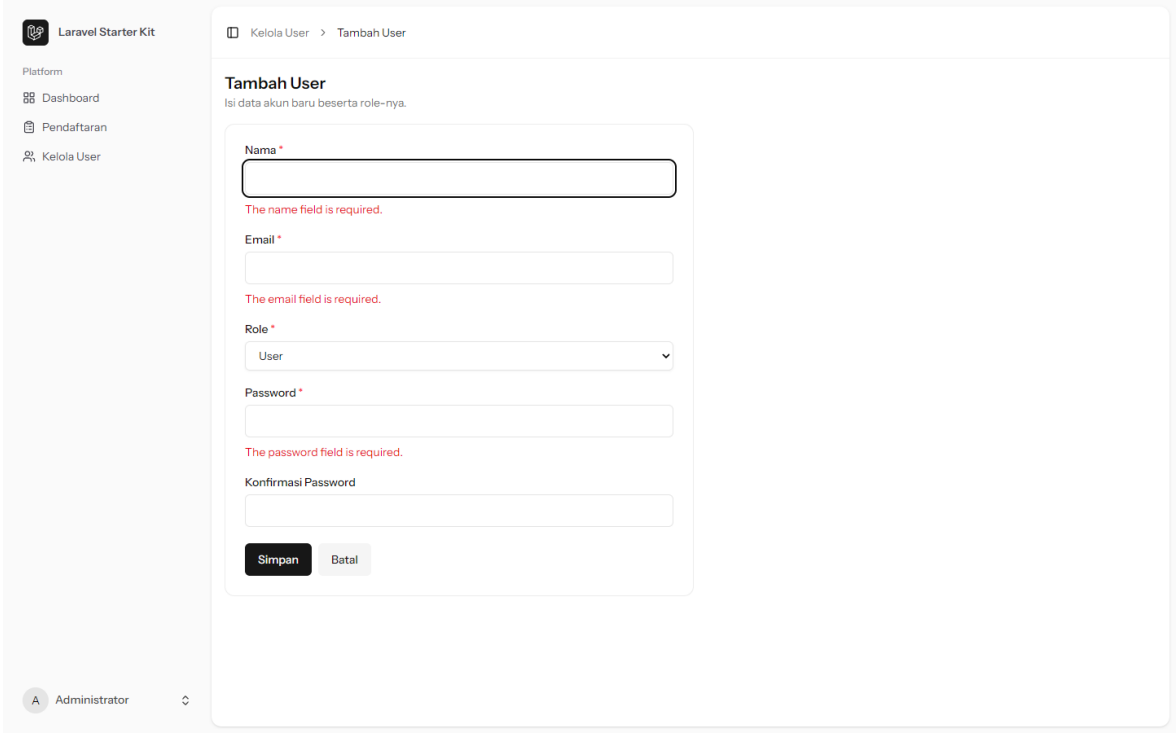
D.3 CRUD User oleh Admin

Halaman Kelola User menampilkan daftar user beserta role-nya. Akun yang sedang login diberi badge Anda dan tidak memiliki tombol Hapus. Pencarian, filter role, dan pagination tetap tersedia di atas tabel.



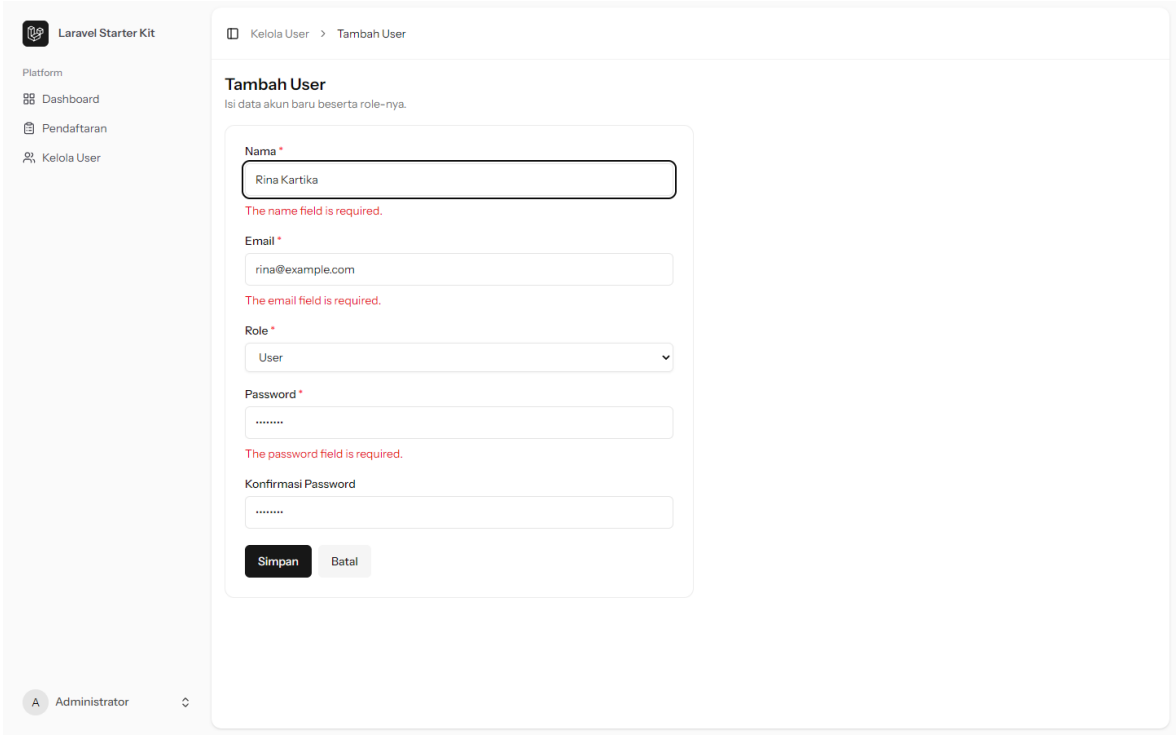
Gambar 9. Daftar user pada halaman Kelola User.

Form tambah user yang dikirim kosong menampilkan pesan galat di bawah setiap input, dan halaman tidak berpindah karena error validasi dikirim kembali sebagai props `errors` milik `useForm`.

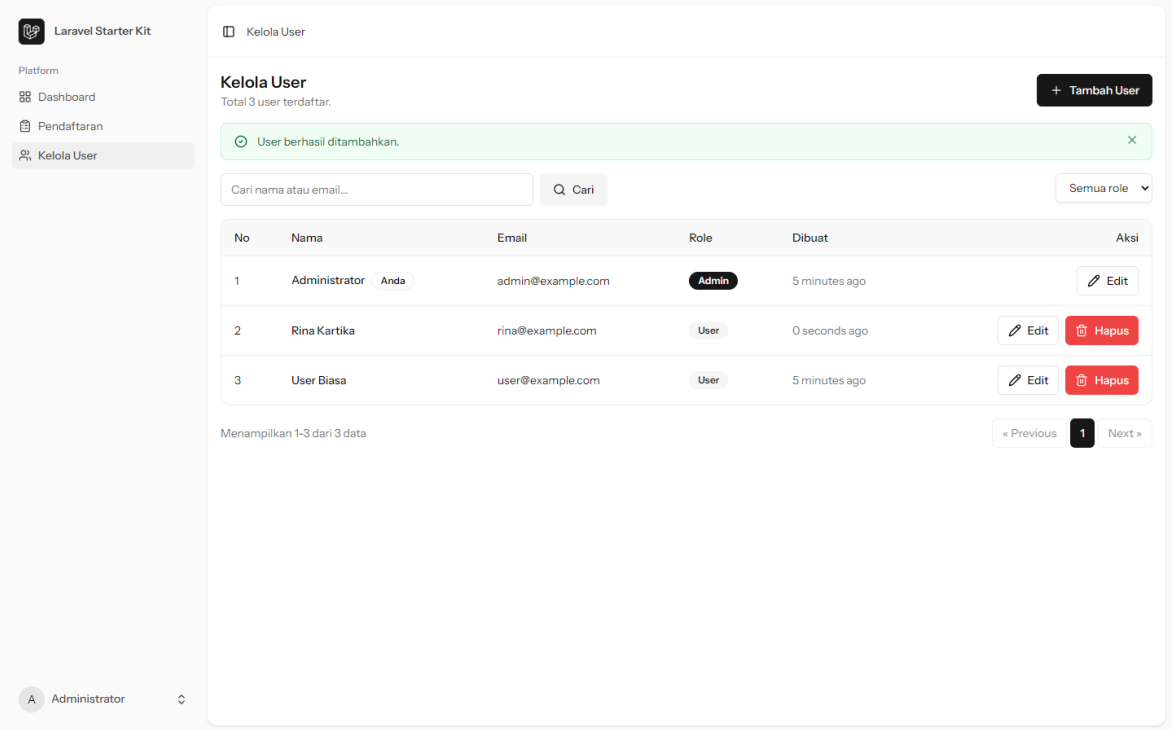


Gambar 10. Galat validasi pada form tambah user.

Setelah data diisi dan disimpan, aplikasi kembali ke daftar user disertai pesan berhasil. User baru langsung muncul pada tabel dan jumlah user pada judul bertambah.

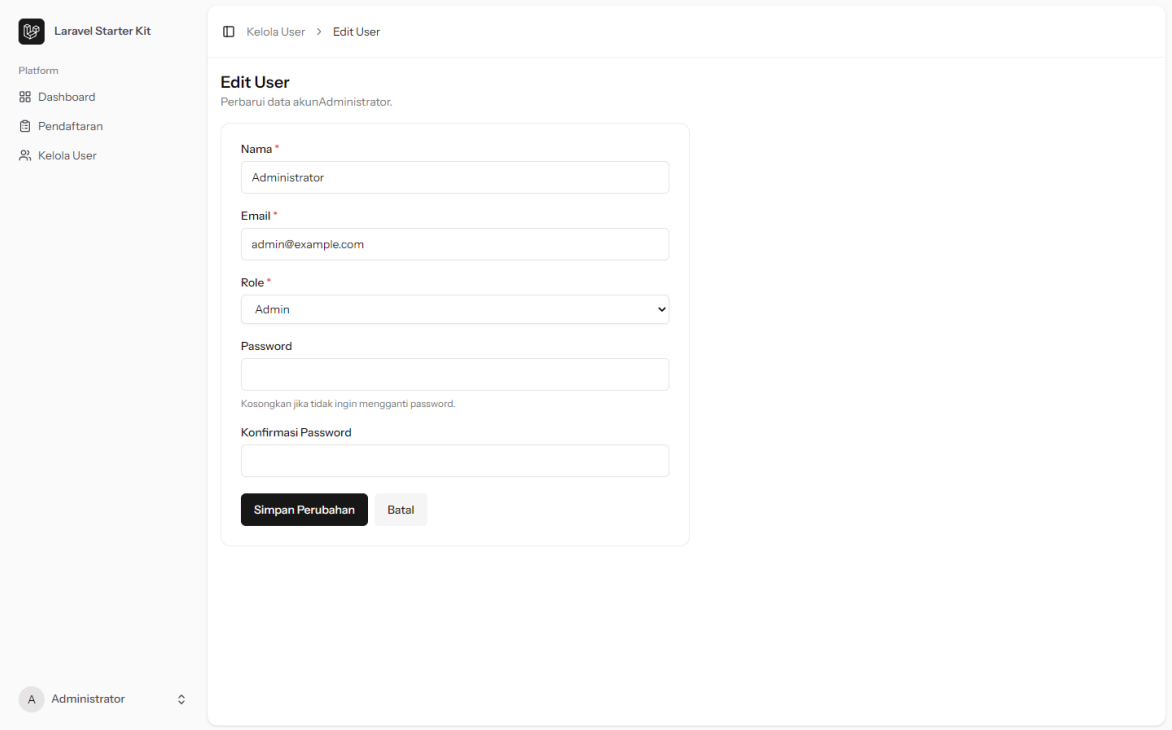


Gambar 11. Form tambah user yang sudah diisi.



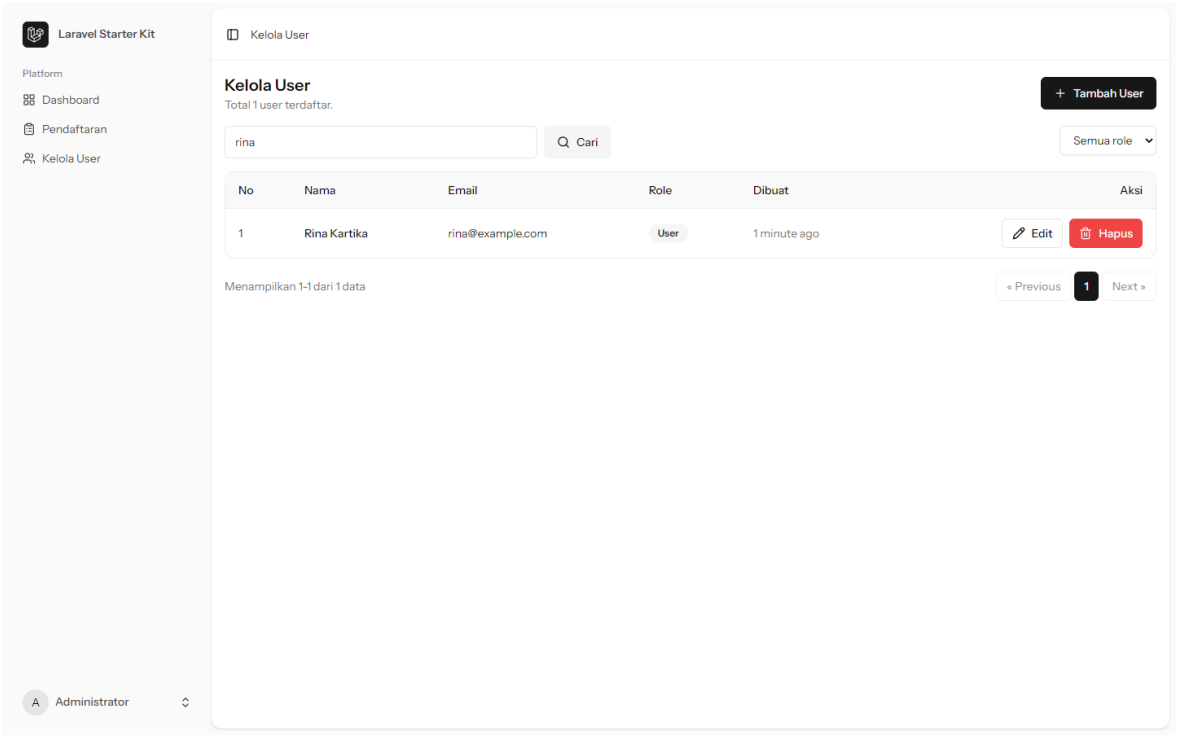
Gambar 12. Pesan berhasil setelah user ditambahkan.

Saat admin mengubah akunnya sendiri, password boleh dikosongkan agar tidak berubah. Pilihan role pada form ini menjadi tidak berbahaya karena `UpdateUserRequest` membatasi nilainya hanya `admin` ketika yang diedit adalah akun sendiri.



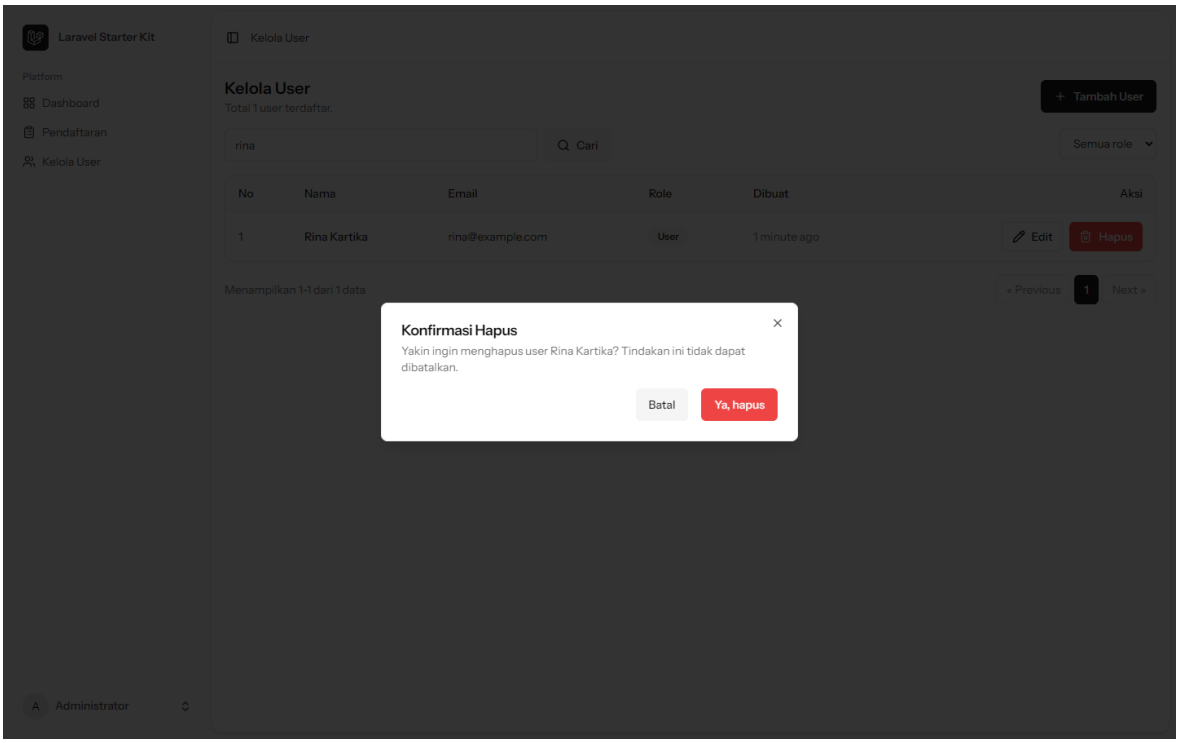
Gambar 13. Form edit akun admin sendiri.

Pencarian memfilter daftar berdasarkan nama atau email melalui query string, sehingga kata kunci tampil pada URL dan dapat dibagikan atau dimuat ulang tanpa kehilangan filter.



Gambar 14. Pencarian user dengan kata kunci.

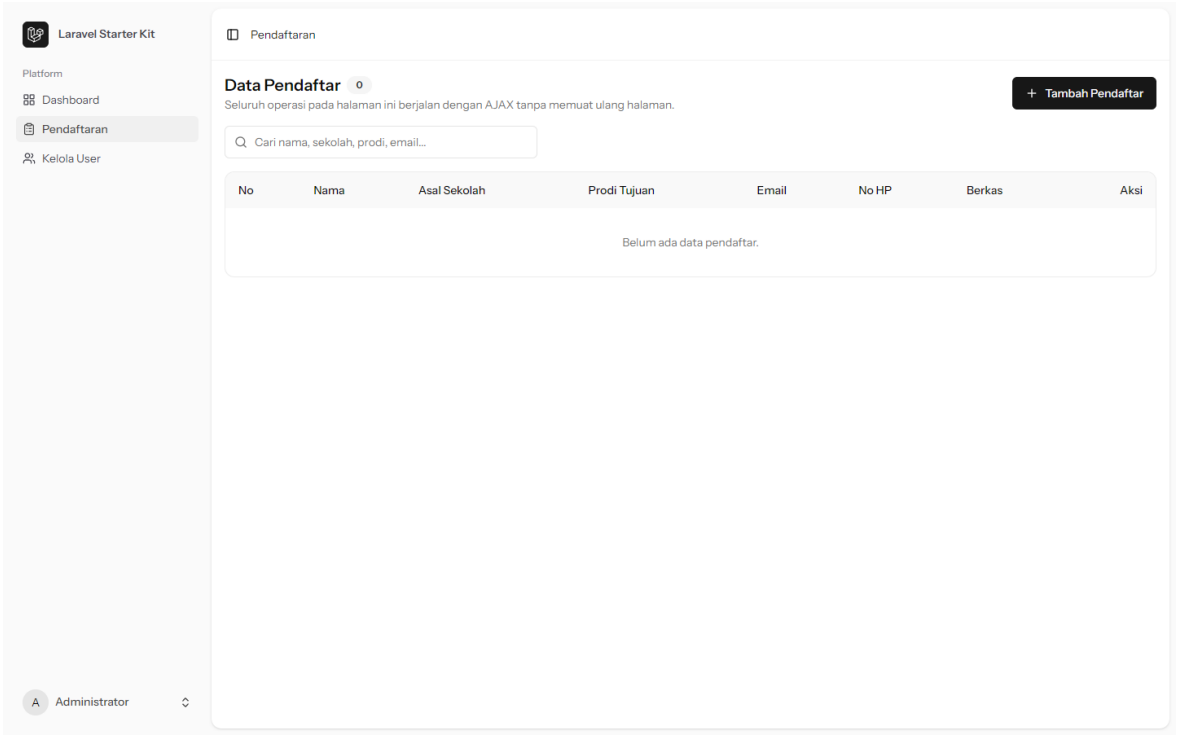
Penghapusan user memakai dialog konfirmasi bersama. Request DELETE baru dikirim setelah pengguna menekan Ya, hapus, sehingga tidak ada penghapusan yang tidak disengaja.



Gambar 15. Dialog konfirmasi sebelum user dihapus.

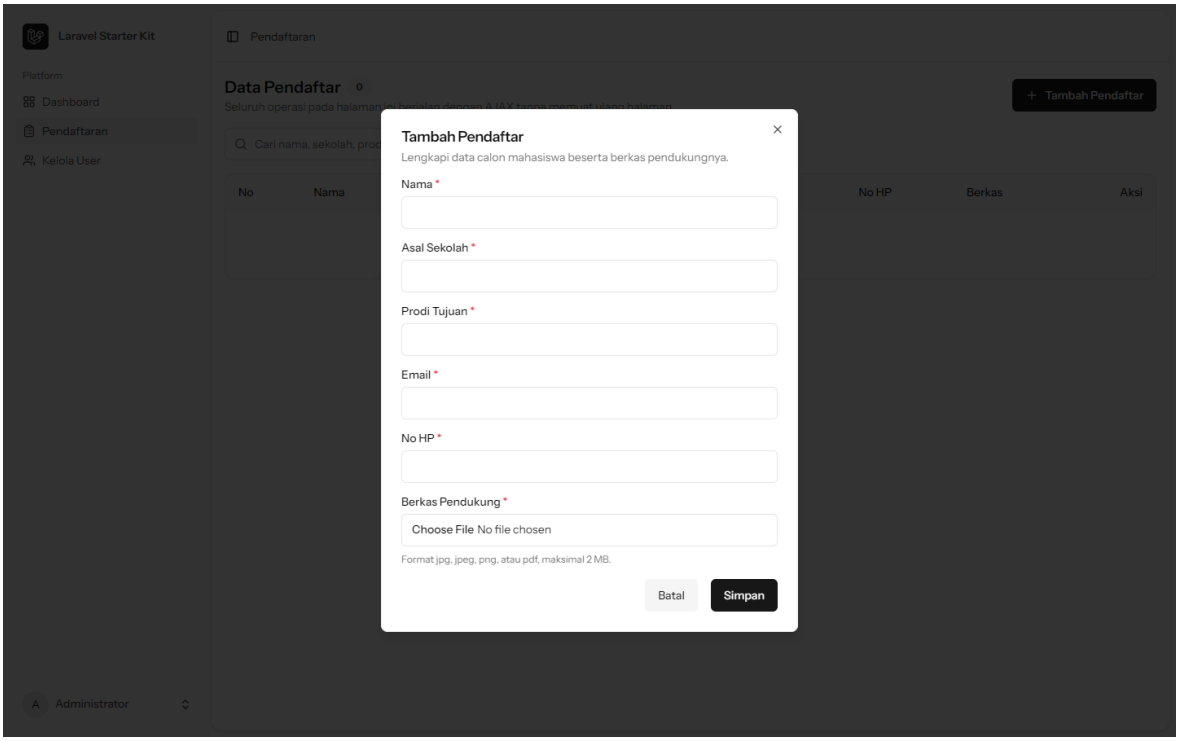
D.4 CRUD Pendaftaran dengan AJAX

Halaman pendaftaran dimuat sekali, lalu isi tabel diambil dari endpoint JSON. Saat belum ada data, tabel menampilkan pesan Belum ada data pendaftar.

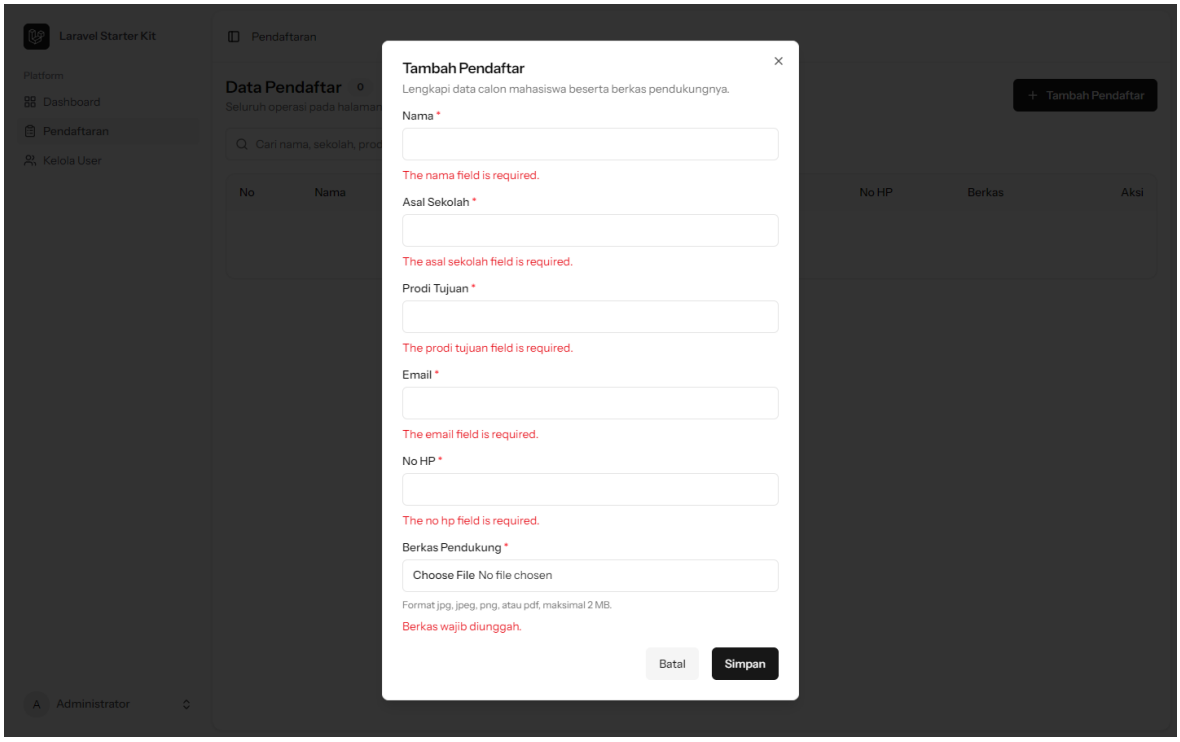


Gambar 16. Halaman Pendaftaran saat belum ada data.

Tombol Tambah Pendaftar membuka dialog form. Bila form dikirim kosong, server mengembalikan 422 dan pesan galat tampil di bawah setiap input tanpa menutup dialog maupun memuat ulang halaman. Pesan untuk berkas memakai teks Bahasa Indonesia dari method `messages()` pada `DaftarRequest`.

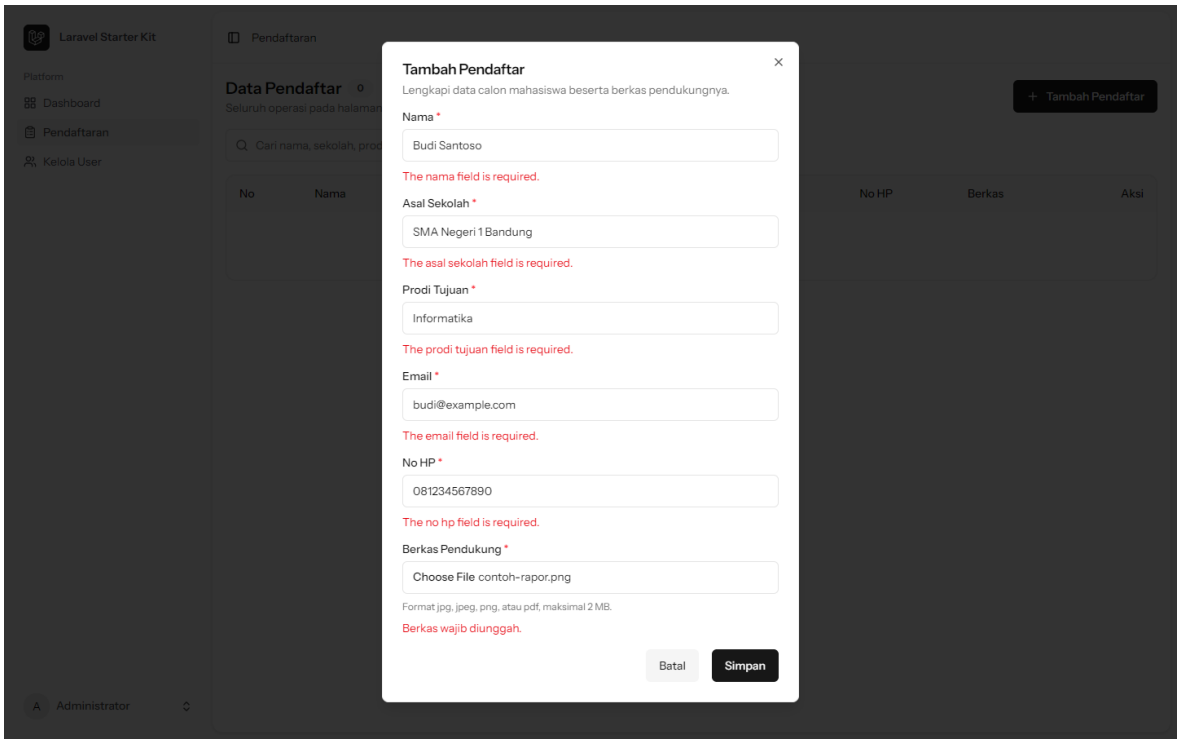


Gambar 17. Dialog tambah pendaftar dalam keadaan kosong.

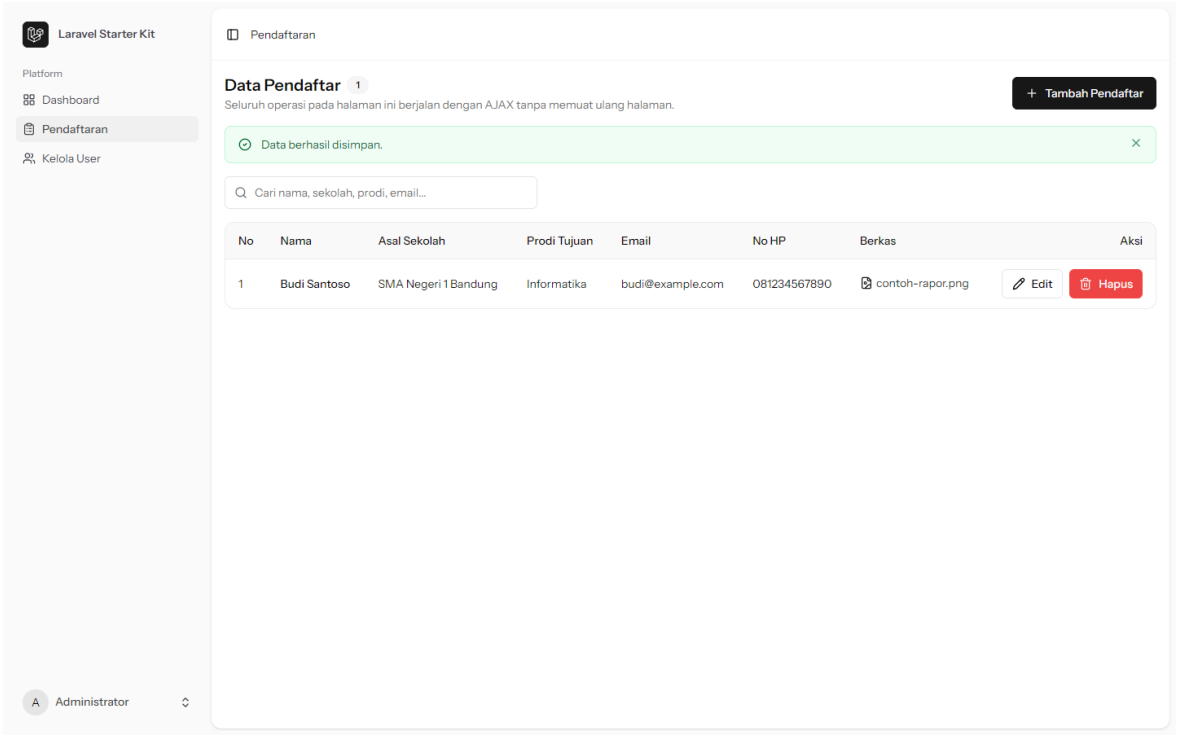


Gambar 18. Respons 422 saat form dikirim kosong.

Setelah seluruh data dan berkas diisi lalu disimpan, dialog tertutup, tabel diperbarui tanpa memuat ulang halaman, dan pesan berhasil muncul di atas tabel.

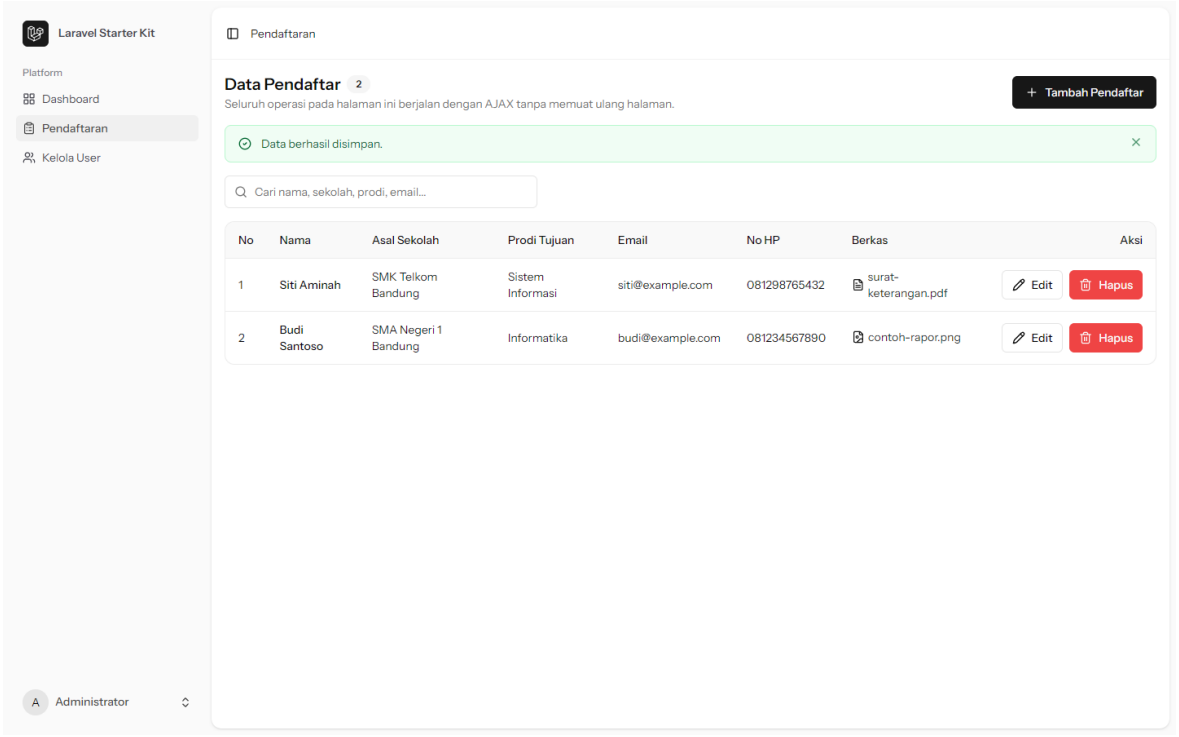


Gambar 19. Dialog tambah pendaftar yang sudah diisi beserta berkas.



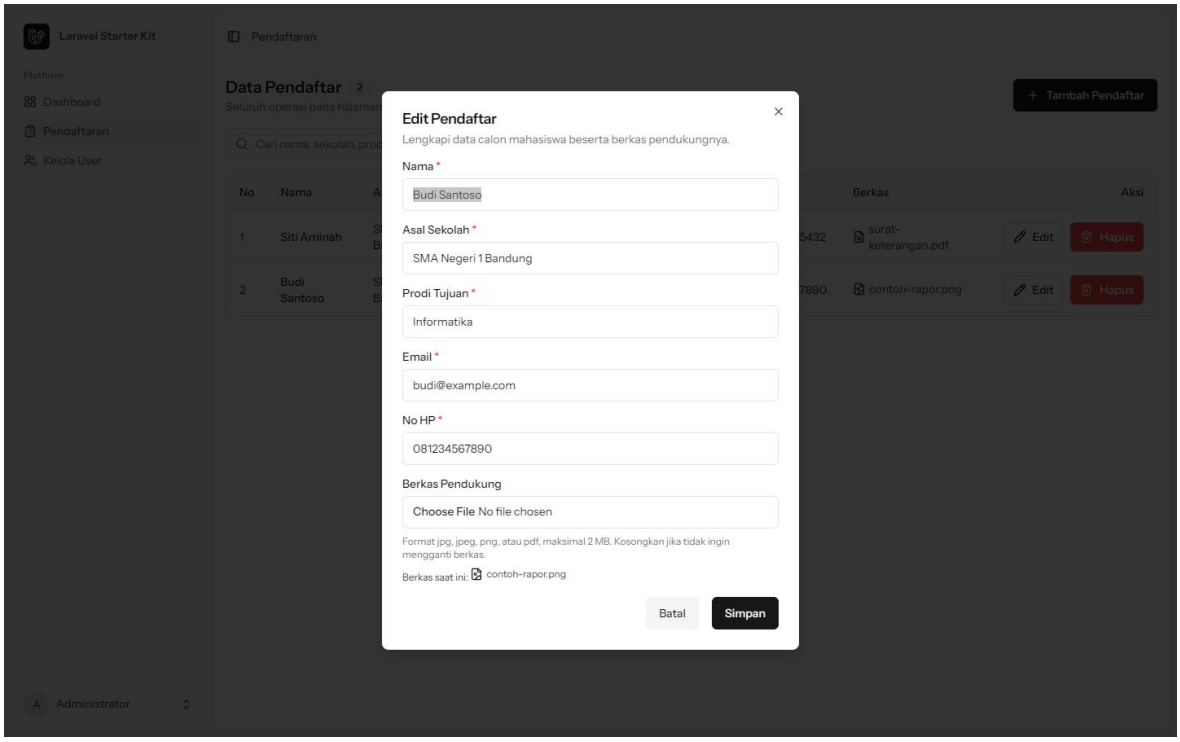
Gambar 20. Pesan berhasil setelah data disimpan lewat AJAX.

Menambah data kedua tidak memuat ulang halaman. Jumlah data pada badge bertambah dan tabel langsung menampilkan dua baris, satu dengan berkas gambar dan satu dengan berkas pdf.



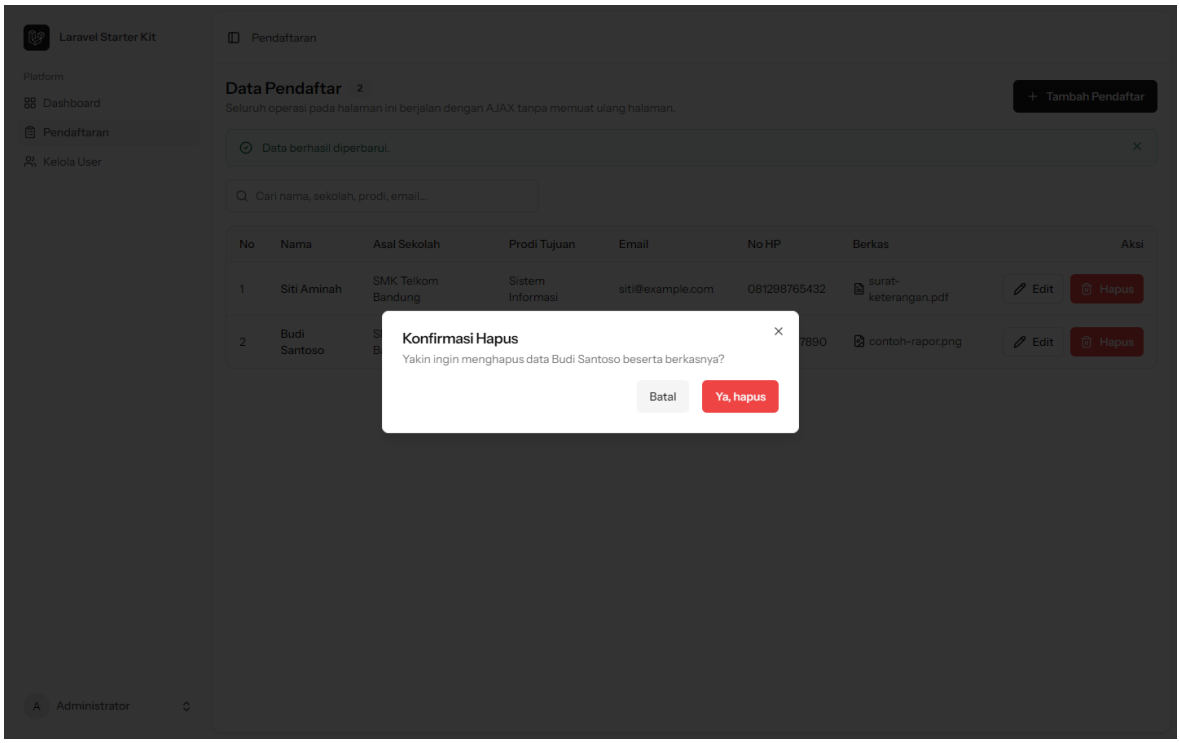
Gambar 21. Daftar pendaftar setelah dua data ditambahkan.

Dialog edit mengambil data lama lewat AJAX dan menampilkan tautan Berkas saat ini. Berkas boleh dikosongkan jika tidak ingin menggantinya, sehingga berkas lama tetap dipakai.



Gambar 22. Dialog edit dengan data lama dan tautan berkas saat ini.

Penghapusan data memakai dialog konfirmasi. Setelah dikonfirmasi, baris hilang dari tabel tanpa berpindah halaman dan berkas milik data tersebut ikut terhapus dari storage.

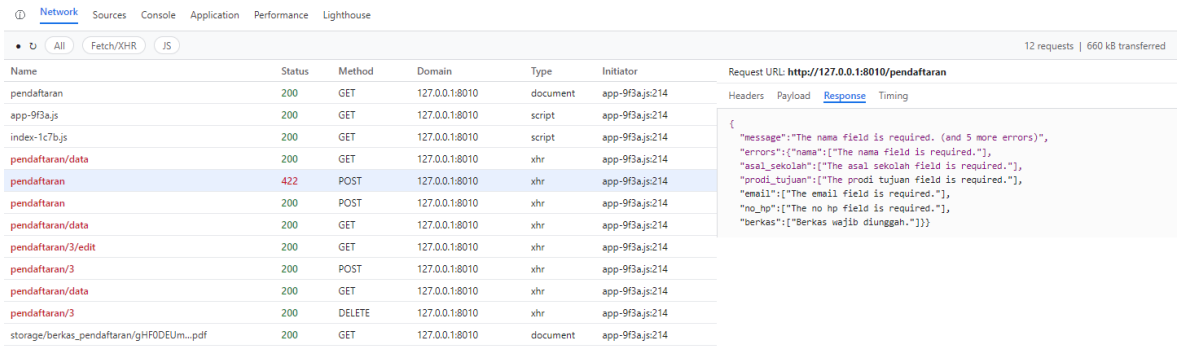


Gambar 23. Dialog konfirmasi sebelum data dihapus.

D.5 Bukti Respons JSON, Network, dan Berkas

Tab Network pada DevTools memperlihatkan bahwa seluruh operasi pendaftaran dikirim sebagai XHR, bukan sebagai navigasi halaman. Terlihat permintaan `GET /pendaftaran/data` untuk memuat tabel, `POST /pendaftaran` dengan status 422 ketika

validasi gagal, `POST /pendaftaran` berstatus 200 ketika data tersimpan, serta `DELETE /pendaftaran/3` ketika data dihapus. Bunyi respons 422 juga tampil pada panel Response beserta daftar kolom yang gagal validasi.



Gambar 24. Tab Network (Fetch/XHR) yang membuktikan CRUD Pendaftaran memakai AJAX.

Endpoint `/pendaftaran/data` mengembalikan array JSON berisi seluruh pendaftar beserta `berkas_url`. Data inilah yang diambil oleh fungsi `loadData()` pada halaman React, lalu disimpan pada state `records`.



Gambar 25. Respons JSON dari endpoint /pendaftaran/data.

Daftar modul diperiksa dengan `php artisan module:list`. Kedua modul berstatus Enabled sehingga route-nya dimuat oleh Laravel.



Gambar 26. Hasil php artisan module:list.

Seluruh route diperiksa dengan `php artisan route:list --except-vendor`. Terlihat enam route `admin/users` dari modul UserManagement dan enam route `pendaftaran` dari modul Pendaftaran, ditambah route dashboard.



Gambar 27. Hasil php artisan route:list --except-vendor.

Pemeriksaan terakhir dilakukan pada folder penyimpanan. Nama berkas di disk berupa hash acak hasil `store()`, sedangkan database hanya menyimpan path dan nama asli berkas.



Gambar 28. Isi folder storage/app/public/berkas_pendaftaran.

E. Analisis dan Pembahasan

E.1 Cara Kerja HMVC dengan Halaman React

HMVC membagi aplikasi menjadi modul, dan setiap modul memiliki controller, form request, model, migration, route, serta halaman React sendiri. Pada project ini modul Pendaftaran dan UserManagement berada di folder `Modules`, sedangkan autentikasi memakai Fortify di aplikasi utama. Namespace kelas mengikuti nama modul, misalnya `Modules\Pendaftaran\Http\Controllers\PendaftaranController`. Composer mengenali namespace tersebut karena `composer.json` setiap modul ikut dibaca melalui `merge-plugin`.

Halaman React milik modul tidak berada di `resources/js/pages`, melainkan di `Modules/*/resources/js/pages`. Karena itu Inertia perlu tahu cara menemukannya. Fungsi `resolve` pada `app.tsx` membaca format `NamaModul::halaman`, memecahnya, lalu memuat berkas yang sesuai lewat `resolvePageComponent`. Vite sendiri mengumpulkan seluruh halaman karena `import.meta.glob` mencakup folder modul, sedangkan Tailwind membutuhkan baris `@source` agar class di halaman modul ikut dibuat.

E.2 Perbedaan Authentication dan Authorization

Authentication menjawab pertanyaan siapa penggunanya, sedangkan authorization menjawab apa yang boleh dilakukannya. Pada praktikum ini authentication sepenuhnya ditangani starter kit beserta halaman React-nya, jadi saya tidak menulis controller login dan register sendiri seperti pada Modul 8. Authorization ditangani middleware `EnsureUserHasRole` yang membaca kolom `role` melalui enum.

Pemisahan keduanya penting. Semua pengguna yang sudah login lolos middleware `auth`, tetapi hanya admin yang lolos middleware `role:admin`. Menu Kelola User pada sidebar hanya menyembunyikan tautan; perlindungan sebenarnya tetap di server, sehingga user yang mengetik `/admin/users` langsung menerima error 403.

E.3 Middleware Role dan Pembatasan Akun Sendiri

Middleware menerima parameter role yang dipisahkan koma. Laravel mengirim bagian setelah titik dua sebagai argumen variadik, lalu middleware memanggil `hasRole()`. Nilai string dari route diubah menjadi enum melalui `UserRole::tryFrom()`, sehingga role yang salah ketik tidak pernah cocok dengan role mana pun.

Pemeriksaan tambahan saya taruh pada `UpdateUserRequest` dan method `destroy`. Admin yang mengubah akunnya sendiri hanya bisa memilih role admin, dan admin tidak bisa menghapus akunnya sendiri. Aturan ini mencegah admin kehilangan akses secara tidak sengaja. Di sisi tampilan, tombol Hapus memang tidak dirender untuk akun sendiri, tetapi aturan di server tetap diperlukan karena request dapat dikirim langsung tanpa melewati tombol tersebut.

E.4 Perbedaan CRUD Inertia dan CRUD AJAX

Aspek	CRUD Inertia (UserManagement)	CRUD AJAX (Pendaftaran)
Respons controller	Inertia::render() atau redirect()->with()	response()->json([...])
Perilaku halaman	Inertia mengganti komponen halaman tanpa full reload; data selalu dikirim ulang server sebagai props	Halaman tidak berpindah; state React diperbarui dari data JSON
Pesan berhasil	Session flash dibagikan sebagai shared props flash	State message diisi dari respons JSON
Error validasi	Redirect kembali; errors otomatis masuk ke useForm()	Kode 422 + JSON errors disimpan ke state errors
Token CSRF	Ditangani otomatis oleh Inertia	Header X-XSRF-TOKEN dari cookie melalui helper http.ts
Form tambah/edit	Halaman terpisah (create, edit) dengan komponen UserForm	Satu Dialog shadcn/ui yang dipakai bergantian
Upload berkas	useForm() otomatis memakai FormData jika data berisi File	FormData manual + fetch() tanpa header Content-Type

Tabel 1. Perbandingan CRUD Inertia dan CRUD AJAX pada kedua modul.

CRUD UserManagement bekerja secara Inertia. Form dikirim lewat `useForm`, server memproses lalu mengembalikan halaman Inertia atau redirect, dan Inertia mengganti komponen halaman tanpa memuat ulang seluruh browser. Data selalu dikirim ulang oleh server sebagai props, dan error validasi otomatis masuk ke objek `errors` milik `useForm`. Pesan berhasil dibagikan sebagai shared props flash.

CRUD Pendaftaran bekerja secara AJAX. Pengiriman form memakai `fetch` dan controller mengembalikan JSON. Tabel diperbarui dengan memanggil `loadData()` tanpa memuat ulang halaman, dan pesan ditampilkan dari respons JSON. Akibatnya, posisi scroll dan isi kotak pencarian tetap terjaga. Konsekuensinya, React harus menangani sendiri status 422 dan mengisi pesan galat ke setiap input, dan itu dilakukan dengan menyalin `error.data.errors` ke state `errors`.

E.5 FormData, Penyimpanan Berkas, dan Token CSRF

Upload berkas tidak bisa memakai JSON biasa karena JSON tidak dapat membawa file. `FormData` dipakai karena objek itulah yang bisa membawa isi input `type=file`. Pada helper `http.ts`, header `Content-Type` sengaja tidak diisi agar browser otomatis memakai `multipart/form-data` beserta boundary. Untuk edit, request tetap dikirim sebagai POST dengan tambahan `_method` bernilai `PUT` karena PHP hanya membaca file pada request POST.

Token CSRF dikirim sebagai header `X-XSRF-TOKEN` yang dibaca dari cookie `XSRF-TOKEN`. Tanpa token ini, Laravel menolak permintaan dengan error 419. Karena request dikirim sebagai AJAX dengan header `Accept: application/json`, kegagalan validasi dikembalikan sebagai JSON berkode 422, bukan sebagai redirect, sehingga helper dapat membacanya.

Berkas disimpan di disk public dengan `store()`, sedangkan database hanya menyimpan path dan nama asli. Pemisahan ini membuat tabel tetap ringan, memungkinkan satu data memiliki berkas yang mudah diganti, dan menyimpan nama asli untuk ditampilkan sebagai tautan. Path hasil `store()` berupa hash acak, sehingga nama berkas pengguna tidak langsung terpakai dan tabrakan nama tidak terjadi. Saat berkas diganti atau data dihapus, berkas lama ikut dihapus dari storage agar tidak ada file sampah.

E.6 Kendala dan Penyelesaian

Beberapa kendala saya temui selama pengerjaan, dan semuanya saya selesaikan tanpa mengubah perilaku aplikasi yang dijanjikan modul.

1. Kelas modul sempat tidak ditemukan karena autoload belum diperbarui setelah modul dibuat. Perbaikannya adalah menjalankan `composer dump-autoload`.
2. Halaman modul sempat gagal dimuat karena template `app.blade.php` masih memuat berkas halaman secara langsung, padahal halaman modul berada di luar `resources/js/pages`. Perbaikannya cukup memuat `app.tsx` pada direktif `@vite`, karena seluruh halaman sudah di-resolve oleh Inertia melalui `import.meta.glob`.
3. Kelas middleware bawaan sempat tidak ditemukan karena berkasnya hilang dari folder `app/Http/Middleware` padahal masih dirujuk `bootstrap/app.php`. Saya membuat ulang kelas tersebut agar request web dapat diproses.
4. Halaman login dan register sempat gagal dirender. Penyebabnya adalah konfigurasi Fortify belum lengkap; setelah view Inertia terdaftar dan batas percobaan login diatur, kedua halaman tampil normal.

5. Koneksi database ditolak dengan pesan `SQLSTATE[HY000] [2002]`. Penyebabnya layanan MariaDB pada XAMPP belum berjalan. Setelah layanan dinyalakan, aplikasi kembali normal.
6. Perintah `npm` sempat diblokir kebijakan eksekusi PowerShell. Saya menjalankannya melalui cmd agar skrip npm dapat dieksekusi.
7. Halaman modul sempat tampil tanpa style karena Tailwind 4 tidak memindai folder modul. Penyelesaiannya adalah menambahkan baris `@source '../../Modules/*/resources/js/**/*.{ts,tsx}'` pada `app.css`.

E.7 Catatan Perbedaan dengan Modul

Ada beberapa perbedaan antara hasil praktikum ini dan contoh pada modul. Saya mencatatnya agar tidak menimbulkan salah paham saat laporan diperiksa.

1. Nama project adalah `pertemuan9_hmvc_react` dengan database `hmvc_react`, sesuai nama pada modul.
2. Seluruh tampilan memakai React Starter Kit dengan Inertia, bukan Blade dan jQuery seperti Modul 8.
3. Kolom data pendaftar mencakup `nama`, `asal_sekolah`, `prodi_tujuan`, `email`, `no_hp`, `berkas`, dan `nama_berkas`, dengan email sebagai kolom unik.
4. Sebagian pesan galat validasi bawaan Laravel pada form AJAX masih berbahasa Inggris, sedangkan pesan untuk no HP dan berkas sudah berbahasa Indonesia lewat method `messages()`.
5. Aplikasi diuji pada alamat `http://127.0.0.1:8010`, bukan `http://127.0.0.1:8000`, karena port 8000 sedang dipakai proses lain.
6. Pengujian Pest masih berisi pengujian bawaan starter kit; pengujian khusus kedua modul belum ditambahkan.

F. Kesimpulan

1. Arsitektur HMVC berhasil diterapkan dengan dua modul, yaitu Pendaftaran dan UserManagement, yang masing-masing memiliki controller, form request, model, migration, route, dan halaman React sendiri di dalam folder `Modules`.
2. Halaman React di dalam modul dapat dimuat melalui Inertia setelah fungsi `resolve` pada `app.tsx` diatur dengan format `NamaModul::halaman`, dan Tailwind CSS ikut membuat class yang dipakai modul setelah baris `@source` ditambahkan.
3. Autentikasi memakai fitur bawaan starter kit, sedangkan pembagian hak akses dilakukan dengan kolom `role`, enum `UserRole`, middleware `role`, dan shared props yang membedakan admin dan user.
4. Middleware role dan aturan tambahan pada form request berhasil membatasi halaman khusus admin, termasuk mencegah admin menurunkan role atau menghapus akunnya sendiri.
5. CRUD UserManagement berjalan secara Inertia dengan form `useForm`, pesan berhasil dari shared props `flash`, dan error validasi yang otomatis tampil di bawah setiap input.

6. CRUD Pendaftaran berjalan dengan `fetch` dan `FormData`, menampilkan error 422 di bawah input, memperbarui tabel tanpa memuat ulang halaman, serta mengelola upload berkas beserta penggantian dan penghapusannya.
7. Pengujian otomatis Pest yang tersedia lulus seluruhnya, yaitu 27 test dengan 64 assertion, sehingga fitur authentication dan dashboard bawaan terbukti berjalan.

Dari praktikum ini saya memahami bahwa HMVC dengan React memisahkan tanggung jawab antar modul tanpa kehilangan pola Laravel, bahwa authentication dan authorization adalah dua hal berbeda, dan bahwa CRUD Inertia maupun AJAX sama-sama menuntut penanganan error validasi serta pesan yang teliti di sisi klien. Saya juga memahami bahwa keamanan halaman tidak boleh bergantung pada tampilan, karena penyembunyian menu hanya kosmetik sedangkan middleware di server adalah penjaga yang sebenarnya.

G. Link Repository GitHub

`https://github.com/Dryhus21/PABW.git`

Repository memuat seluruh kode program, termasuk folder `Modules` beserta halaman React di dalamnya, middleware, controller, form request, route, seeder, serta komponen di `resources/js`. Folder `node_modules` dan `vendor` tidak diunggah.